VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY

HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY

MAI CHÍ BẢO

Investigating a Hybrid LLM-GNN Model to Enhance the Efficiency of ADAPT-QAOA for Quantum Circuit Optimization

Major: Computer Science

Major code: 8480101

MASTER'S THESIS

HO CHI MINH CITY, May 2026

THIS THESIS IS COMPLETED AT
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY - VNU-HCM

Supervisor 1: Assoc. Prof. Dr. Thoại Nam

Signature: .....

Supervisor 2: .....

Signature: .....

Examiner 1: .....

Signature: .....

Examiner 2: .....

Signature: .....

This master's thesis is defended at HCM City University of Technology, VNU-HCM City on June 08, 2026.

Master's Thesis Committee:

1. **Chair:** .....
2. **Secretary:** .....
3. **Member:** .....
4. **Examiner 1:** .....
5. **Examiner 2:** .....

Approval of the Chair of Master's Thesis Committee and Dean of Faculty of Computer Science and Engineering after the thesis being corrected.

CHAIR OF THESIS COMMITTEE

(Full name and signature)

DEAN OF FACULTY

OF COMPUTER SCIENCE AND ENGINEERING

(Full name and signature)

VIETNAM NATIONAL UNIVERSITY - HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY

SOCIALIST REPUBLIC OF VIETNAM
Independence - Freedom - Happiness

THE TASK SHEET OF MASTER'S THESIS

Full name: MAI CHÍ BẢO **Student ID:** 2370691

Date of birth: December, 24, 1999 **Place of birth:** Pleiku City, Gia Lai

Major: Computer Science **Major ID:** 8480101

I. THESIS TITLE (*In Vietnamese*):

Nghiên cứu mô hình lai LLM-GNN nhằm nâng cao hiệu quả ADAPT-QAOA trong tối ưu hóa mạch lượng tử

II. THESIS TITLE (*In English*):

Investigating a Hybrid LLM-GNN Model to Enhance the Efficiency of ADAPT-QAOA for Quantum Circuit Optimization

III. TASKS AND CONTENTS:

Task	Content
Literature Review	Reviewing quantum optimization (QAOA) and hybrid algorithms. Identifying critical gaps in existing methods
Performance Evaluation	Completely conducting the experimental results of the cutting-edge methods with a comprehensive evaluation.
Research Gap Analysis	Finding and demonstrating the research gap in previous approaches.
Idea Implementation	Proposing and implementing the Hybrid LLM-GNN assisted ADAPT-QAOA framework.
Report	Writing manuscript for thesis report.

IV. THESIS START DAY: January 12, 2026

V. THESIS COMPLETION DAY: May 17, 2026

VI. SUPERVISOR:

- **Supervisor 1:** Assoc. Prof. Dr. Thoại Nam

Ho Chi Minh City, May 10, 2026

SUPERVISOR	**HEAD OF DEPARTMENT**
(Full name and signature)	*(Full name and signature)*

DEAN OF FACULTY OF COMPUTER SCIENCE AND ENGINEERING

(Full name and signature)

ACKNOWLEDGEMENTS

First and foremost, I would like to express my sincere gratitude to *Assoc. Prof. Dr. Thoại Nam* for his invaluable guidance throughout my master's journey. I am deeply thankful for his dedicated mentorship, insightful advice, and continuous support in helping me navigate both academic and professional challenges.

I would also like to extend my heartfelt appreciation to all lecturers at Ho Chi Minh City University of Technology for their dedication and commitment to teaching. Their efforts in delivering valuable knowledge have provided me with a strong foundation for both my studies and future career.

I would like to express my deepest gratitude to my family for their constant encouragement and emotional support throughout this journey. Their belief in me has been a strong source of motivation. I am also sincerely thankful to my loved one for understanding the nature of my work and always being there during both the challenging and rewarding moments.

I would like to extend a special appreciation to the computing resources that made this research possible: my MSI laptop and my desktop PC (NVIDIA GeForce RTX 3060, 12GB VRAM). These machines worked tirelessly through long days and nights to generate datasets and train models, playing an essential role in completing this thesis.

Last but not least "*I thank me for believing in me, I want to thank me for doing all this hard work, I wanna thank me for having no days off and I wanna thank me for never quitting"*

Ho Chi Minh City, May 10, 2026
Master Student

Mai Chí Bảo

ABSTRACT

Combinatorial optimization problems play a critical role in real-world applications such as logistics and network design, yet remain challenging for classical methods as problem size grows. Hybrid quantum - classical algorithms, particularly the Quantum Approximate Optimization Algorithm (QAOA), offer a promising alternative. Its adaptive variant further improves flexibility by dynamically constructing problem-specific circuits. However, its effectiveness is limited by two key challenges: generating efficient circuit structures and initializing parameters to ensure stable and fast convergence.

In addition, existing approaches often treat circuit generation and parameter initialization separately, leading to limited generalization across different circuit depths and reduced performance on weighted graphs. They also lack a unified mechanism to incorporate graph structural information into both processes.

To address these issues, this thesis proposes a unified generative - predictive framework for adaptive QAOA. Graph embeddings derived from NetLSD, FEATHER, and Graph Neural Networks (GNNs) are used to encode structural information, which conditions a transformer-based model (NanoGPT and LLaMA) to autoregressively generate quantum circuits. By integrating circuit structure and parameter information into a single sequence representation, the framework enables joint optimization and improves generalization across different configurations.

Experimental results on weighted and unweighted Max-Cut datasets demonstrate stable training behavior, rapid convergence, and high approximation quality. The model also learns to reduce circuit depth while maintaining performance, highlighting its ability to balance solution quality and computational efficiency.

Overall, this thesis presents a scalable and unified approach to adaptive QAOA, improving both circuit generation and optimization performance.

THE COMMITMENT OF THE THESIS' AUTHOR

I hereby declare that this thesis is my own original work, conducted under the scientific supervision of *Assoc. Prof. Dr. Thoại Nam*. The research contents and results presented in this thesis are authentic and represent the research work conducted solely by myself throughout the course of my Master's degree program at Ho Chi Minh City University of Technology. All referenced sources and materials have been properly cited and acknowledged.

I take full responsibility for the content of my research.

Ho Chi Minh City, May 10, 2026
Master Student

Mai Chí Bảo

CONTENTS

List of Figures

List of Tables

cases where valid circuits were not generated for all instances. The best and second-best values in each row are highlighted in bold and underlined, respectively. Methods proposed in this work are additionally indicated with a green background. 75

List of Abbreviations

AI	Artificial Intelligence
QAOA	Quantum Approximate Optimization Algorithm
ADAPT-QAOA	Adaptive Quantum Approximate Optimization Algorithm
LLM	Large Language Model
NetLSD	Network Laplacian Spectral Descriptor
FEATHER	Fast and Efficient Attributed graph Embedding via spectral decomposition
GNN	Graph Neural Networks
ER	Erdős - Rényi
GPT	Generative Pre-trained Transformer
RMSNorm	Root Mean Square Layer Normalization
RoPE	Rotary Positional Embedding
SwiGLU	Swish-Gated Linear Unit
GPU	Graphics Processing Unit
CPU	Central Processing Unit
AR	Approximation Ratio
VQE	Variational Quantum Eigensolver
PCA	Principal Component Analysis
GCN	Graph Convolutional Network
GAT	Graph Attention Network
GIN	Graph Isomorphism Network
GraphSAGE	Graph Sample and Aggregate

CHAPTER 1. INTRODUCTION

1.1 Problem Statement

Modern real-world applications such as logistics optimization, network design, and resource allocation require solving large-scale combinatorial optimization problems with high efficiency. As these problems grow in size and complexity, classical algorithms increasingly struggle to provide high-quality solutions within reasonable computational time, creating a demand for more powerful computational paradigms.

Quantum computing has emerged as a promising direction to address these challenges, particularly through hybrid quantum-classical algorithms [1][2] that can leverage near-term quantum devices. Among these, the Quantum Approximate Optimization Algorithm (QAOA) [17] has gained significant attention due to its flexibility and applicability to problems such as Max-Cut. Its adaptive variant further enhances this capability by dynamically constructing problem-specific quantum circuits, offering the potential for improved performance and resource efficiency.

Despite these advantages, the practical effectiveness of adaptive QAOA [1][2][7] remains limited by several fundamental challenges:

- **Circuit structure generation challenge:** Designing suitable circuit structures remains difficult due to the exponentially growing ansatz search space. Existing methods often rely on heuristics or manually designed strategies, limiting scalability and adaptability to different graph structures.
- **Parameter initialization challenge:** Initialization strongly affects optimization performance. Poor initialization can cause barren plateaus or slow convergence, while existing learning-based methods often ignore circuit depth and physical principles, leading to unstable results across configurations.

- **Generalization and scalability limitation:** Current approaches show limited generalization across varying QAOA depths and weighted graph instances. More fundamentally, they lack a unified framework that jointly models circuit structure, parameter initialization, and problem characteristics in a scalable and adaptive way.

These limitations raise key questions: how to automatically generate efficient graph-aware adaptive QAOA circuits, learn parameter initialization for fast and stable convergence, and generalize across different circuit depths and problem settings while maintaining solution quality and circuit efficiency. Addressing these challenges is essential for applying adaptive QAOA to real-world optimization problems on near-term quantum hardware.

1.2 Thesis Overview

As discussed in Section 1.1, solving combinatorial optimization problems with adaptive QAOA requires the coordinated interaction of multiple components within a unified pipeline. In general, such a system can be decomposed into four essential elements: *(i)* the graph representation that encodes structural properties of the problem instance, *(ii)* the circuit generation mechanism that constructs the quantum ansatz, *(iii)* the parameter initialization strategy that guides the optimization process, and *(iv)* the training objective that governs learning dynamics and solution quality. These components collectively determine the effectiveness, scalability, and generalization capability of adaptive QAOA systems.

Among these components, circuit generation and parameter initialization play a particularly critical role, as they directly define the search space and optimization landscape of the problem. An inefficient circuit structure leads to unnecessary computational overhead, while poor parameter initialization can result in barren plateaus or slow convergence. Therefore, an effective adaptive QAOA framework must jointly optimize both structure and parameters in a coherent and scalable manner.

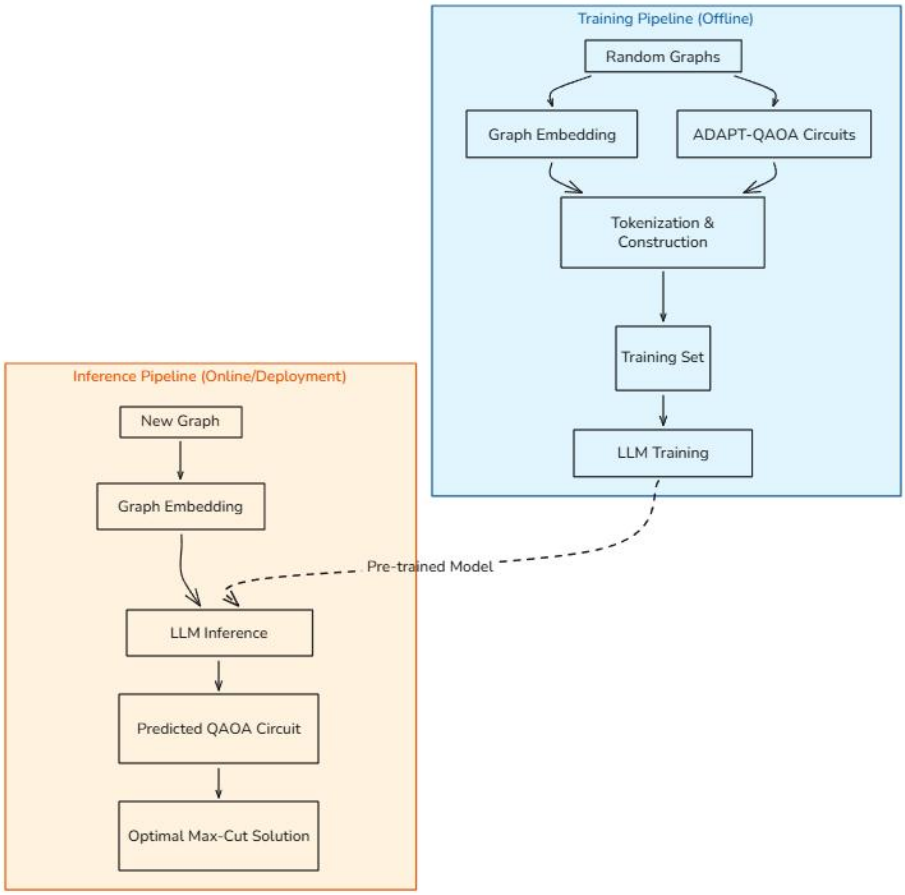

Figure 1-1 Proposed Hybrid LLM-GNN Framework Assisted ADAPT-QAOA for Optimizing Graph-Structured Quantum Circuits in this Thesis

Existing approaches often treat circuit construction and parameter initialization separately, relying on heuristic designs, handcrafted rules, or independent learning models. This separation limits adaptability, weakens generalization across graph structures and circuit depths, and fails to consistently capture global structural information during circuit generation.

To address these limitations, this thesis proposes a unified generative-predictive framework that combines graph representation learning, generative modeling, and physics-informed parameter prediction into a single system. Adaptive QAOA circuit construction is formulated as a graph-conditioned sequence generation problem, where both circuit structures and initialization parameters are learned jointly.

The framework contains two pipelines: Training Flow and Inference Flow. During training, diverse weighted and unweighted Max-Cut graphs are generated, and high-quality reference circuits are obtained using ADAPT-QAOA. Graphs are encoded into embeddings using methods such as NetLSD, FEATHER, or GNNs, then integrated into a graph-conditioned transformer that autoregressively generates quantum circuit sequences. Circuit depth and parameter information are incorporated into the token representation, enabling the model to learn scalable and physics-informed initialization strategies.

Unlike conventional methods, the proposed framework jointly optimizes circuit generation and parameter prediction within a single autoregressive model, enabling adaptive complexity reduction while maintaining solution quality. Experimental results show stable training, fast convergence, improved approximation performance, and reduced circuit depth. Overall, this thesis introduces a scalable and unified perspective on adaptive QAOA through graph-conditioned sequence generation.

1.3 Contributions

The primary contributions of this thesis can be summarized as follows:

- This work investigates the learning dynamics of a generative - predictive framework for adaptive QAOA by training NanoGPT and LLaMA models on graph representations derived from different embedding methods like NetLSD, FEATHER or GNN. The proposed approach demonstrates rapid convergence behavior, where the Approximation Ratio (AR) increases sharply and exceeds 0.9 within early training iterations, while the Error Rate (ER) correspondingly decreases and stabilizes to zero after sufficient training
- A joint optimization mechanism is implicitly achieved, where both solution quality and circuit structure are improved during training. In particular, the model learns to reduce circuit complexity, decreasing the number of QAOA

layers from an initial high value to a significantly more compact representation while maintaining high performance

- The proposed framework integrates generative modeling with graph-based feature representations, enabling the transformation of structural graph information into adaptive QAOA circuits. This design provides a novel approach for leveraging learned embeddings to guide both circuit construction and parameter initialization
- An analysis of structural convergence is conducted, showing that the number of circuit layers stabilizes after training, indicating that the model can consistently identify efficient circuit configurations without further growth in complexity.
- Extensive experimental evaluation on a synthetic dataset demonstrates that the proposed approach achieves stable training behavior across multiple metrics, including approximation ratio, error rate, and circuit depth, confirming its effectiveness and robustness for adaptive QAOA optimization

1.4 Thesis Structure

The remainder of this thesis is structured as follows:

- **Chapter 2: Related Work** reviews existing approaches in quantum optimization and hybrid quantum - classical algorithms, with a focus on QAOA and its adaptive variants. The chapter analyzes prior methods for circuit construction and parameter initialization.
- **Chapter 3: Preliminaries** establishes the theoretical foundations required for this work. This chapter introduces the Max-Cut problem, the formulation of QAOA, and the adaptive QAOA (ADAPT-QAOA) procedure. It also presents essential concepts in graph representation learning, including spectral embeddings and graph neural networks, as well as an overview of transformer-based generative models for sequence modeling.

- **Chapter 4: Methodology** presents the proposed hybrid LLM-GNN assisted ADAPT-QAOA framework. It describes the overall training and inference pipelines, graph embedding methods, ADAPT-QAOA circuit generation, and the graph-conditioned tokenization process. The chapter also details the transformer-based models used for circuit generation, including how structural and circuit information are incorporated into sequence modeling.
- **Chapter 5: Experimental Environment** describes the dataset and experimental setup. It covers the construction of weighted and unweighted graph datasets, hardware configuration, and model settings. The chapter also outlines the training procedure and evaluation metrics, including approximation ratio, circuit validity, validation loss, and circuit depth.
- **Chapter 6: Results & Analysis** presents experimental results and detailed analysis of the proposed framework. The chapter evaluates the impact of different graph embeddings and model architectures on performance, analyzes training dynamics such as convergence behavior and error reduction, and examines the relationship between approximation quality and circuit complexity. It also provides comparative results demonstrating the effectiveness and stability of the proposed approach across different settings.
- **Chapter 7: Conclusions & Future Work** summarizes the main contributions of the thesis and it's significance for quantum circuit optimization using learning-based approaches. It provides an overview of the proposed framework and its role in integrating graph representation learning with sequence modeling. The chapter also outlines several directions for future research.

CHAPTER 2. RELATED WORK

2.1 Learning-Based Enhancements for QAOA

Early efforts to improve the Quantum Approximate Optimization Algorithm focus on reducing the cost of parameter optimization, which is one of its main bottlenecks. A prominent direction is to replace random initialization with learned parameter prediction.

Works such as Graph Learning for Parameter Prediction of QAOA [9] and Iterative-Free QAOA Using Neural Networks [13] demonstrate that neural networks can learn a mapping from graph structure to high-quality parameters (β, γ). These approaches treat QAOA as a supervised learning problem, where optimized parameters from prior runs serve as training labels. As a result, QAOA can converge faster and, in some cases, eliminate iterative optimization entirely.

Similarly, QSeer [5] extends this idea by incorporating quantum-inspired constraints into the learning process. By restricting parameter ranges and enforcing physically motivated trends (e.g., adiabatic evolution), QSeer improves both training stability and generalization across graph types and circuit depths. Unlike earlier works, it supports both weighted graphs and variable-depth QAOA within a unified framework.

Overall, these methods shift part of the optimization burden from quantum hardware to classical learning models, significantly improving efficiency. However, they are limited to parameter prediction within fixed circuit structures, rather than generating circuits themselves.

2.2 Graph Neural Networks for Max-Cut and QAOA Initialization

Graph Neural Networks (GNNs) have been widely used to capture structural patterns in combinatorial optimization problems such as Max-Cut. In the context of QAOA, GNNs are primarily used as warm-start mechanisms.

For instance, Graph Neural Network Initialisation of QAOA [14] proposes an unsupervised GNN that learns node-level assignments corresponding to Max-Cut partitions. These outputs are then used to construct better initial quantum states, guiding QAOA toward promising regions of the solution space. This approach avoids the need for labeled optimal solutions while still achieving strong generalization.

Other works explore different GNN architectures (e.g., GCN, GAT, GIN, GraphSAGE) to learn graph-level embeddings that correlate with optimal QAOA parameters. These embeddings serve as compact representations of graph structure and are critical for scaling learning-based methods to larger instances. Despite their effectiveness, these approaches primarily focus on improving initialization or parameter prediction, rather than addressing the design of the quantum circuit itself.

2.3 Generative Models for Quantum Circuit Design

A more recent research direction reframes quantum optimization as a sequence generation problem, leveraging generative models to construct quantum circuits directly. The Generative Quantum Eigensolver [16] (implemented via GPT-QE) replaces parameter optimization with a probabilistic model over quantum circuits. Instead of tuning parameters in a fixed ansatz (as in VQE or QAOA), the model generates sequences of quantum gates and is trained to favor low-energy circuits using a reinforcement-learning-like objective. This approach introduces a more flexible and potentially smoother optimization landscape.

Building on similar ideas, QAOA-GPT [4] applies a decoder-only Transformer to generate QAOA circuits conditioned on graph embeddings. By encoding graphs (via methods such as FEATHER) and training on graph–circuit pairs produced by ADAPT-QAOA, the model learns to directly map problem instances to circuit structures. This represents a significant shift from parameter learning to full circuit synthesis.

Additionally, GroverGPT [15] demonstrates that large language models can approximate quantum algorithms themselves, learning patterns of quantum behavior without explicit simulation. While focused on Grover's algorithm rather than QAOA, it highlights the broader potential of LLMs in quantum computing tasks. These works collectively show that generative models, especially Transformer-based architectures, are powerful tools for exploring the space of quantum circuits.

2.4 Incremental and Adaptive Learning Strategies in QAOA

Another line of work focuses on improving scalability and training efficiency through adaptive or incremental strategies. The PIL-QAOA [12] method introduces a progressive learning framework in which QAOA is first trained on small subgraphs and then extended to larger graphs by reusing previously learned parameters. This reduces optimization complexity and improves convergence stability, especially for large-scale problems.

Such approaches highlight the importance of curriculum learning and parameter reuse in quantum optimization, particularly when dealing with exponentially growing search spaces. However, these methods still operate within predefined circuit structures and do not leverage modern generative models or language-based representations.

2.5 Summary

From the above literature, several important limitations can be identified that highlight gaps in current research. First, most learning-based approaches primarily focus on predicting good initialization parameters for QAOA, typically expressed as (β, γ), rather than addressing the more challenging problem of generating the quantum circuit structure itself. While these methods significantly improve convergence speed and solution quality, they remain constrained within predefined circuit ansatz and do not explore the broader design space of adaptive quantum circuits.

Second, recent generative approaches such as Generative Quantum Eigensolver and QAOA-GPT show promising results by reformulating quantum optimization as a sequence generation problem. However, these methods are still limited in how effectively they incorporate rich structural information from input graphs. In many cases, graph representations are either simplified or insufficiently integrated into the generative process, which may restrict their ability to generalize across diverse problem instances.

Third, although Graph Neural Network-based methods are highly effective at encoding graph structure into meaningful representations, their integration with large language model-based generative frameworks remains limited. Existing works tend to treat GNNs and generative models as separate components, without fully leveraging their complementary strengths in structural understanding and sequence modeling.

Finally, only a small number of studies explicitly address the generation of ADAPT-QAOA circuits, where the circuit is constructed dynamically through operator selection rather than relying on a fixed ansatz. This setting introduces additional complexity, as the model must learn not only parameter values but also the structure and ordering of quantum operations. As a result, the problem of learning to generate adaptive quantum circuits from graph-structured inputs remains largely underexplored.

CHAPTER 3. PRELIMINARIES

This chapter introduces the fundamental concepts and theoretical background underlying this thesis. It covers key techniques from quantum optimization, large language models, and graph representation learning, which together form the foundation of the proposed approach.

Section 3.1 presents ADAPT-QAOA, an extension of QAOA that constructs the variational ansatz adaptively through gradient-based operator selection, improving optimization efficiency and convergence. Section 3.2 provides an overview of large language models (LLMs) and their capability to model structured sequences, including quantum circuits. Section 3.3 discusses graph-level embeddings, comparing spectral methods and graph neural networks in terms of representation power and scalability. Section 3.4 introduces the Max-Cut problem in the context of QAOA and defines key evaluation metrics such as expected energy and approximation ratio.

3.1 Adaptive Derivative-Assembled Problem-Tailored Quantum Approximate Optimization Algorithm (ADAPT-QAOA).

An important advancement in variational quantum algorithms is the Adaptive Derivative-Assembled Problem-Tailored Quantum Approximate Optimization Algorithm (ADAPT-QAOA) [7]. This method replaces the fixed mixer Hamiltonian used in standard QAOA with an automatically constructed, problem-adaptive ansatz by progressively adding operators from a predefined operator pool. Inspired by the ADAPT-VQE approach, ADAPT-QAOA addresses several limitations of traditional QAOA [4][5][7], including slow convergence and the increasing classical optimization cost as the number of parameters grows.

In ADAPT-QAOA, the ansatz is expanded layer by layer using a gradient-based operator selection rule. At each iteration k, candidate o perators A_j from the mixer pool are evaluated using the energy gradient.

$$g_j = \frac{\partial E}{\partial \beta_j} = -i\langle \psi^{(k-1)} | e^{i\gamma_0 H_c} [H_c, A_j] e^{-i\gamma_0 H_c} | \psi^{(k-1)} \rangle$$

Where

- H_c: cost Hamiltonian
- γ_0 : represents the initial variational parameters associated with the problem Hamiltonian.
- g_j: measures how sensitive the energy is to each candidate operator

The operator with the largest g_j value is selected and added to the ansatz, producing a new quantum state.

$$|\psi^{(k)}\rangle = e^{-i\beta_k A^{(k)}} e^{-i\gamma_k H_c} |\psi^{(k-1)}\rangle$$

The operator pool may include single-qubit Pauli operators such as (X_i, Y_i), their linear combinations, or two-qubit entangling operators such as $X_i Z_j$ or $Y_i Y_j$.

After adding a new layer, all variational parameters are re-optimized to minimize the expectation value of the cost Hamiltonian.

$$\langle \psi^{(k)} | H_c | \psi^{(k)} \rangle$$

The iterative process continues until the gradient falls below a predefined threshold, indicating that further expansion of the ansatz no longer provides significant improvement.

3.2 Large Language Models (LLMs)

Large language models (LLMs) [3][18] based on the transformer architecture have become a powerful foundation for learning from large-scale unlabeled text datasets. These models typically use either decoder-only or encoder-decoder transformer architectures with multiple stacked self-attention layers, enabling them to capture long-range dependencies and generate sequences in an autoregressive or sequence-to-sequence manner.

During the pre-training stage, the model is trained on continuous text data using a token prediction task, where it learns to estimate the probability of a token given its surrounding context.

Given a token sequence $U = \{u_1, u_2, ..., u_n\}$, an autoregressive LLM is optimized by maximizing the log-likelihood of each token conditioned on the preceding tokens.

$$\mathcal{L}_{LM}(U) = \sum_{i=1}^{n} \log P(u_i | u_1, u_2, ..., u_{i-1}; \Theta)$$

Here, θ represents the model parameters. Input tokens are mapped into token embeddings combined with positional encodings, allowing the model to capture both semantic meaning and sequential order. The masked self-attention mechanism restricts predictions to the permitted contextual positions.

The token-based sequence generation framework of LLMs can be naturally extended beyond language domains. Structured sequences such as source code, mathematical expressions, or quantum circuit descriptions can all be represented in tokenized form. In particular, quantum circuits (essentially sequences of unitary operations) are well suited to sequence-based modeling [3].

Therefore, LLMs provide a principled approach to generating, optimizing, and exploring new quantum circuit structures by learning patterns from sufficiently large and diverse datasets. The objective of this project is to evaluate the scalability of such models for these types of problems and to investigate whether their generative modeling capabilities can be leveraged to discover more efficient quantum circuit architectures.

3.3 Graph Level Embeddings

Graph-level embeddings map a graph $G = (V, E)$ into a fixed-dimensional vector that summarizes its structure and properties. In this thesis, three main approaches are considered: NetLSD, FEATHER and GNN-based embeddings.

More generally, graph embedding refers to the process of transforming a graph, which is inherently non-Euclidean and irregular, into a continuous vector space where standard machine learning models can be applied. The goal is to encode both local and global structural information, such as node connectivity, edge weights, and overall topology, into a compact representation. This transformation enables efficient comparison between graphs, supports downstream tasks such as classification or generation, and allows models like transformers to process graph-structured data. Formally, a graph embedding can be defined as a function $f: G \rightarrow R^d$, where each graph is mapped to a vector in a d-dimensional space while preserving meaningful structural similarities.

3.3.1 Network Laplacian Spectral Descriptor (NetLSD)

NetLSD (Network Laplacian Spectral Descriptor) [11] is a graph embedding method that converts a graph into a fixed-length numerical vector. Instead of analyzing nodes or edges directly, it studies how information (or heat) spreads through the graph over time. By observing this diffusion process at multiple time scales, it captures both: Local structure (how nodes connect in small neighborhoods) and Global structure (overall topology and connectivity pattern). The final result is a numerical signature that represents the structural identity of the graph.

How NetLSD works:

Step 1: Compute the Normalized Laplacian Matrix

NetLSD begins by computing the normalized Laplacian matrix from the graph adjacency matrix. This matrix mathematically represents the graph structure,

capturing connectivity patterns while remaining scale-invariant and numerically stable. For weighted graphs, edge weights are directly incorporated into the Laplacian, allowing the representation to reflect connection strengths as well as topology.

Step 2: Compute Eigenvalues of the Laplacian

The eigenvalues of the normalized Laplacian are computed to capture important structural characteristics of the graph. Small eigenvalues mainly describe global connectivity patterns, while larger eigenvalues capture local structural variations. Using the full eigenvalue spectrum provides a precise and informative representation of the graph structure.

Step 3: Define Diffusion Time Scales

NetLSD then analyzes graph diffusion across multiple time scales by defining lower and upper diffusion bounds together with the number of scale steps. These parameters determine the observation range and resolution of the diffusion process. At small diffusion times, heat remains close to the starting node, emphasizing local graph structure, whereas at larger diffusion times, heat spreads throughout the graph and captures global structural information

Step 4: Compute Heat Trace at Each Scale

For each time scale t, NetLSD computes:

$$h(t) = \sum_i e^{-t\lambda_i}$$

Where:

- λ_i are the Laplacian eigenvalues
- h(t) measures how much heat remains in the graph at time t

Each time scale produces one numerical value.

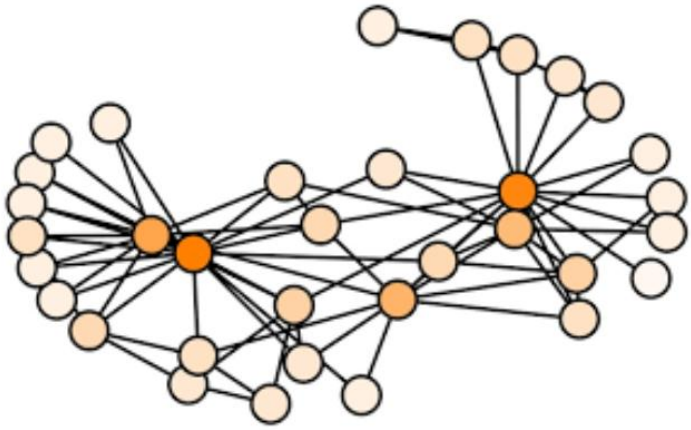

Figure 3-1 Visualization of NetLSD method as [11]

3.3.2 Fast and Efficient Attributed graph Embedding via spectral decomposition (FEATHER)

FEATHER (Fast Evaluation of Attributes Through Heat-kernel-like Embedding Representation) [10] is an unsupervised graph embedding method that generates structural representations based on node features and multi-scale propagation. The method combines node-level structural descriptors, characteristic function transformations using sine and cosine encodings, and iterative propagation through the normalized adjacency matrix.

It captures how node features diffuse across the graph structure and aggregates them into a fixed-length embedding vector. The final result is a numerical representation that reflects both local node characteristics and multi-hop structural influence.

How FEATHER Works

Step 1: Create Normalized Adjacency Matrix

FEATHER computes:

$$A = D^{-1}A$$

Where A is the adjacency matrix and D^{-1} is the inverse degree matrix. The normalization process ensures scale invariance, stable feature propagation, and

compatibility with both weighted and unweighted graphs. Self-loops are added so that each node preserves and contributes its own features during propagation.

Step 2: Construct Initial Node Features

For each node, two structural features are computed:

- Log degree: $\log(\text{degree} + 1)$
- Clustering coefficient

These features capture node connectivity strength and local triangle density, providing a structural representation before propagation.

Step 3: Characteristic Function Transformation

FEATHER evaluates the features at multiple points:

$$\theta \in [0.01, \theta_{max}\ ]$$

For each feature and each evaluation point. Apply cosine and sine transformation

$$\cos(X\theta), \sin(X\theta)$$

This expands the feature space, captures distributional properties, and improves representation expressiveness without requiring additional learning.

Step 4: Multi-Hop Propagation

Features are propagated through the normalized adjacency matrix:

$$X^{(k)} = AX^{(k-1)}$$

This is repeated for a fixed number of steps (order).

Each propagation step captures:

- 1-hop neighborhood influence
- 2-hop influence
- Up to "k"-hop structural context

All propagated feature blocks are concatenated.

Step 5: Graph-Level Pooling

Since FEATHER is originally designed for node-level representations, node embeddings are aggregated into a graph-level embedding using mean pooling by default. This produces a fixed-size vector representation for the entire graph.

3.3.3 Graph Neural Networks (GNN)

Graph neural networks (GNNs) [6] generate graph-level embeddings through a message-passing mechanism. At each step, node features are updated based on information aggregated from their neighboring nodes. After this process, all node embeddings are combined using permutation-invariant functions such as sum, mean, or max to produce a single vector representing the entire graph.

One major advantage of GNNs is their end-to-end trainability. Unlike handcrafted graph descriptors, GNNs can directly optimize embeddings for a target objective while leveraging both node and edge features. They are also highly flexible, supporting weighted and unweighted graphs as well as heterogeneous graph structures. Furthermore, deep message passing enables GNNs to capture higher-order interactions and long-range dependencies that are difficult to model using traditional graph embedding techniques.

However, GNNs also introduce several challenges. Training GNNs requires significantly higher computational resources due to repeated neighborhood aggregation and parameter optimization. As the number of layers increases, GNNs may suffer from oversmoothing, where node embeddings become overly similar and lose discriminative power. Deep architectures can also experience vanishing gradients and unstable optimization. Despite these limitations, GNNs remain one of the most powerful and widely used approaches for learning expressive graph representations. Table 3-1 presents a comparative overview of the graph embedding methods GNN, NetLSD, and FEATHER across multiple criteria.

Criteria	NetLSD	Feather	GNNs
Need Training?	No	No	Yes (end-to-end)
Structural Focus	Mainly Global (multi-scale Laplacian spectrum)	Global + Local (spectral + attributes)	Local → Global (multi-hop message passing)
Supports Weighted Graphs	Yes (via weighted Laplacian)	Yes	Yes
Supports Node Attributes	No	Yes	Yes
Supports Edge Attributes	No	Limited	Yes
Computational Speed	Fast	Fast	Slower
Scalability	Very scalable	Scalable	Depends on model depth and graph size
Optimization Capability	Fixed embedding (no task adaptation)	Fixed embedding	Task-adaptive representation

Table 3-1 Graph embeddings comparision: GNN, NetLSD and FEATHER

3.4 QAOA for Max-Cut Problem

The Quantum Approximate Optimization Algorithm (QAOA) is a well-suited method for solving combinatorial optimization problems, with Max-Cut being one of its most representative applications [8]. The Max-Cut problem aims to find a bipartition of a graph such that the number (or total weight) of edges connecting the two subsets is maximized. For a graph $G = (V, E)$, the problem is encoded into a cost Hamiltonian:

$$H_z = \frac{1}{2}\sum_{(j,k)\in E} w_{j,k}\left(1 - Z_j Z_k\right)$$

Here:

- Z_j is the Pauli-Z operator acting on qubit j.
- j, k is the weight of the edge j, k.

The unweighted case corresponds to $w_{j,k} = 1$ for all edges. The average absolute edge weight is defined accordingly.

$$\overline{|w|} = \frac{1}{|E|}\sum_{(j,k)\in E} |w_{j,k}|$$

Each operator $Z_j Z_k$ in the Hamiltonian acts on two qubits j, k, corresponding to an edge (j, k) in the graph. The performance of QAOA at depth p is evaluated through the expected energy:

$$F_p(\gamma, \beta) \equiv \langle \psi_p(\gamma, \beta) | H_z | \psi_p(\gamma, \beta) \rangle$$

where (γ, β) are the variational parameters. Solving Max-Cut with QAOA corresponds to finding the set of parameters that optimizes this expectation value.

$$(\gamma^*, \beta^*) \equiv arg \max_{\gamma,\beta} F(\gamma, \beta)$$

An important metric used to evaluate solution quality is the approximation ratio [7]:

$$\alpha \equiv \frac{F(\gamma^*, \beta^*)}{C_{\max}}$$

where C_{max} is the maximum possible Max-Cut value of the graph. This ratio satisfies $0 \leq \alpha \leq 1$ and $\alpha = 1$ corresponding to the optimal solution.

Increasing the QAOA depth p typically improves the approximation ratio α. However, as the circuit becomes deeper, the model may suffer from the barren plateau phenomenon, where gradients decrease exponentially and approach zero. In such cases, optimization can easily get stuck if parameters are initialized randomly. Therefore, effective parameter initialization strategies become crucial for enabling QAOA to converge to high-quality solutions, especially as circuit depth increases

CHAPTER 4. METHODOLOGY

This chapter presents the proposed graph-conditioned quantum circuit generation framework using a hybrid LLM-GNN assisted ADAPT-QAOA approach. Section 4.1 provides an overview of the training and inference pipelines. Section 4.2 describes graph dataset preparation for both weighted and unweighted graphs, including high-quality ADAPT-QAOA solutions. Section 4.3 introduces the graph embedding methods, including NetLSD, FEATHER, and GNN-based embeddings. Section 4.4 explains the ADAPT-QAOA circuit generation process, while Section 4.5 details tokenization and training dataset construction. Finally, Section 4.6 presents the graph-conditioned transformer architectures, forming a complete pipeline for generating efficient quantum circuits from graph structures.

4.1 Hybrid LLM-GNN Assisted ADAPT-QAOA Framework

This work proposes a hybrid framework that integrates ADAPT-QAOA, graph representation learning, and large language models (LLMs) to enable data-driven quantum circuit generation for the Max-Cut problem. The key idea is to treat quantum circuits as structured sequences and leverage sequence modeling techniques to learn the relationship between graph structure and effective QAOA ansatz design.

The motivation for combining these components is threefold. First, ADAPT-QAOA provides a principled method for constructing high-quality quantum circuits through iterative, gradient-based operator selection. However, it requires repeated optimization for each new graph, leading to high computational cost. Second, graph embeddings (e.g., GNN, NetLSD, FEATHER) offer compact representations that capture both local and global structural properties of graphs, enabling the model to condition circuit generation on graph-specific characteristics. Third, LLMs are highly effective at modeling complex sequential patterns, making them well-suited

for representing quantum circuits as token sequences and learning generation rules from data.

Within this framework, the input is a graph $G = (V, E)$, which may be either weighted or unweighted, and the output is a parameterized quantum circuit tailored to the given graph. The framework learns to approximate the mapping from graph structure to high-quality ADAPT-QAOA circuits by training on graph-circuit pairs generated offline.

The overall system is organized into two main pipelines: the training flow and the inference flow. The training flow constructs a supervised dataset by generating diverse graphs, computing their corresponding ADAPT-QAOA circuits, graph embeddings, and encoding both graph structure and circuits into token sequences. A graph-conditioned transformer model is then trained to predict circuit tokens autoregressively.

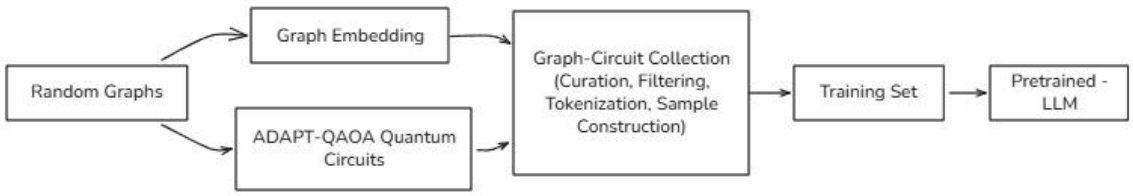

Figure 4-1 Training pipeline consists of 5 main stages: (1) Graph Dataset Preparation, (2) Graph Embedding Generation, (3) ADAPT-QAOA Circuit Generation, (4) Tokenization & Training Set Construction, (5) Model Training

In contrast, the inference flow is designed for efficient deployment. Given a new graph, the trained model directly generates a quantum circuit without requiring iterative optimization.

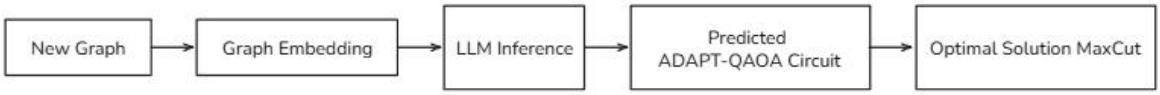

Figure 4-2 Inference pipeline consists of 3 stages: (1) Create Graph Embedding, (2) Generate ADAPT-QAOA circuit using LLM, (3) Create Optimal Max-cut Solution from the created circuits

By combining adaptive quantum optimization, graph-aware representations, and sequence modeling, the proposed framework enables scalable and generalizable quantum circuit generation across a wide range of graph instances.

4.2 Graph Dataset Preparation

To train and evaluate learning models for ADAPT-QAOA, a graph dataset is constructed to cover a wide range of structures, sizes, and weight configurations. The goal is to include both unweighted and weighted Max-Cut problems, while providing high-quality reference labels, including QAOA cost values, approximation ratios, and optimal initialization parameters - across multiple circuit depths.

The dataset primarily consists of weighted ER graphs with node sizes $n \in \{9,10,11\}$. For each graph, edges are generated with varying densities, and weights are sampled from the uniform distribution $U(0,1)$. This setting allows the dataset to capture a wide range of weighted Max-Cut instances while maintaining a consistent and controlled distribution of edge weights.

In addition to weighted graphs, a small subset of unweighted graphs is also included. These graphs can be interpreted as a special case of the weighted setting, where all edge weights are equal to 1. Although limited in number, they provide additional structural diversity and serve as a reference for simpler problem instances.

All graphs in the dataset are solved using ADAPT-QAOA with circuit depths up to $p \leq n$, where n is the number of nodes. To ensure robust optimization, each instance is trained using multiple random initializations, reducing the likelihood of convergence to poor local minima. The resulting solutions include optimized parameter sets, final energy values, and corresponding approximation ratios.

To guarantee the quality of the dataset, a filtering step is applied based on the approximation ratio. Specifically, only graph instances achieving approximation ratio $\phi \geq 0.97$ are retained. This threshold ensures that the generated circuits closely

approximate optimal Max-Cut solutions, making them reliable as supervised learning targets.

4.3 Graph Embeddings

The graphs considered in this thesis have the following properties:

- Small size $n \in \{9,10,11\}$
- Both weighted and unweighted structures
- Requirement of stable fixed-length embeddings

4.3.1 NetLSD Embedding

Build the Embedding Vector

If 500 time scales are sampled, the resulting embedding vector is:

$$[h(t_1), h(t_2), \ldots, h(t_{500})]$$

This produces a 500-dimensional embedding vector. The number of scale steps directly determines the embedding dimension.

Parameter Selection

Based on these characteristics, the following NetLSD parameters are selected:

- **Lower Scale Bound = -2**
  Focuses on fine local connectivity, captures small structural differences between similar small graphs, and is important because small graphs have limited structural complexity.
- **Upper Scale Bound = 2**
  Captures global topology, ensures diffusion fully propagates across the entire graph, and helps distinguish overall connectivity patterns.
- **Scale Steps = 500**
  Directly determines the embedding dimension, provides high-resolution

structural description across diffusion scales, and ensures the required 500-dimensional output vector.

- **Approximations = 50**
  Mainly affects computational speed for large graphs. Since the graphs in this thesis contain only 9 - 11 nodes, the full eigenvalue spectrum is computed, and approximation has negligible impact.

Advantages of NetLSD

- Produces fixed-size, stable, and deterministic embeddings without requiring training.
- Captures both local and global graph structures through multi-scale spectral analysis.
- Naturally supports weighted and unweighted graphs and works effectively for small graphs where eigenvalues can be computed accurately.

Limitations of NetLSD

- For small graphs, heat trace values may become extremely small and lose information after rounding.
- Strong focus on global structure can produce highly similar embeddings for small graphs with limited variation.
- Does not learn task-specific representations and may perform better on larger graphs than small-scale settings.

4.3.2 FEATHER Embedding

Parameter Selection

Based on these characteristics, the following FEATHER parameters are selected:

- **FEATHER_ORDER = 5**
  Captures up to 5-hop neighborhood information, which is sufficient to cover

the full structure of small graphs (9 - 11 nodes). Higher orders would introduce redundancy.

- **FEATHER_EVAL_POINTS = 25**
  Defines the number of characteristic function evaluation points. Provides balanced resolution of feature transformation without excessive dimensional growth.
- **FEATHER_THETA_MAX = 2.5**
  Controls the upper range of evaluation points $\theta \in [0.01, \theta_{\max}]$. Ensures meaningful variation in sine/cosine encoding while maintaining numerical stability for small feature magnitudes.
- **FEATHER_POOLING = mean**
  Aggregates node embeddings by averaging, ensuring stable and size-independent graph representations. This is suitable for small graphs with limited node variation.

Advantages of FEATHER

- Produces fixed-size embeddings efficiently without requiring training.
- Captures both node-level and multi-hop structural information through feature propagation.
- Naturally supports weighted and unweighted graphs and works effectively for small graphs.

Limitations of FEATHER

- Relies on handcrafted node features, which may limit representation expressiveness.
- Does not explicitly capture global spectral structure as effectively as NetLSD.
- Does not learn task-specific representations like GNNs, and pooling may reduce structural variation for very small graphs.

4.3.3 GNN Embedding

This thesis uses a Graph Neural Network (GNN), specifically a graph-level embedding model based on the Graph Isomorphism Network (GIN). Unlike NetLSD and FEATHER, which rely on fixed structural descriptors, the proposed GNN is trainable and learns task-relevant graph representations directly from data. The model generates node embeddings through iterative message passing, where each node updates its representation using information aggregated from neighboring nodes. These node representations are then combined into a graph-level embedding, producing a fixed 500-dimensional vector representation for each graph.

The GNN is trained using contrastive self-supervised learning, enabling the model to learn meaningful structural patterns without requiring explicit labels. As a result, the final embeddings capture structural, weighted, and feature-based characteristics learned from the synthetic graph dataset.

4.3.3.1 GNN Architecture

Each node starts with a 4-dimensional feature vector. These features are passed through 3 GIN layers (hidden size 128), where nodes aggregate information from their neighbors to learn structural representations. Dropout is applied after each layer for regularization.

The updated node embeddings are then combined using attentional pooling, which learns how much each node should contribute to the graph representation. Finally, a linear projection maps the pooled vector to a fixed 500-dimensional graph embedding used for downstream tasks.

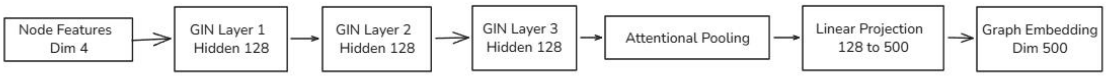

Figure 4-3 Proposed GNN-based architecture for generating graph-level embeddings. The model consists of three GIN layers, followed by attentional pooling and a linear projection to a 500-dimensional embedding space

4.3.3.2 Training Strategy

Unlike embedding methods such as NetLSD and FEATHER, which are deterministic and do not require learning, the Graph Neural Network (GNN) in this thesis is explicitly trained using a self-supervised contrastive learning framework. This approach allows the model to learn meaningful graph representations directly from data without relying on labeled supervision.

To construct the training dataset, synthetic connected ER graphs are generated. Ensuring connectivity is important, as it guarantees effective message passing across the entire graph and avoids isolated components that could degrade representation quality. Edge weights are assigned using either a uniform distribution in the range $[0,1.0]$, allowing the model to encounter both moderately varying and highly skewed weight patterns. For node features, each node is initialized with a random vector sampled from a normal distribution with dimension 4, providing a simple but flexible feature space for learning.

To support contrastive learning, each graph is transformed into two different augmented views. These augmentations are designed to introduce small but meaningful perturbations while preserving the overall structure of the graph. Specifically, edge dropout randomly removes a subset of edges, simulating partial structural changes. Edge weight noise slightly perturbs the weights of existing edges, capturing uncertainty or variability in edge strength. In addition, node feature noise is applied by injecting small random variations into node attributes.

Through these augmentations, the model is trained to produce embeddings that remain consistent across different views of the same underlying graph. In other words, the GNN learns to focus on invariant structural properties rather than superficial differences, resulting in robust and generalizable graph representations.

4.3.3.3 Contrastive Learning Objective

A temperature-scaled contrastive loss is employed to train the model in a self-supervised manner. During training, two augmented versions of the same graph are

first generated, and each version is passed through the network to produce corresponding embeddings, denoted as z_1 and z_2. These embeddings are then normalized to ensure stable similarity computation. The training objective is to maximize the similarity between embeddings derived from the same graph (positive pairs), while simultaneously minimizing the similarity between embeddings of different graphs within the same batch (negative pairs).

The loss function is defined as:

$$L = CrossEntropy\left(\frac{z_1 z_2^T}{T}\right)$$

Where $T = 0.2$ is the temperature parameter that controls the concentration level of the similarity distribution. A lower temperature sharpens the distinction between positive and negative pairs, making the model more sensitive to differences. Overall, this objective encourages embeddings of the same graph to be close to each other in the representation space, while pushing embeddings of different graphs farther apart, thereby improving the discriminative power of the learned representations.

4.4 Circuit Generation (ADAPT - QAOA)

This stage generates quantum circuits using an adaptive ansatz to obtain high-quality approximate solutions for the Max-Cut problem.

The ADAPT-QAOA procedure works as follows [7]:

1) Initialize the parameters γ_o associated with the problem Hamiltonian.
2) At each iteration k, select the operator $O_k \in \mathcal{P}$ from a fixed operator pool that has the largest gradient.

$$\mathcal{P} = \{O_1, O_2, ..., O_m\}$$

3) Add the selected operator to the ansatz.

$$\{\gamma_1, \beta_1, ..., \gamma_k, \beta_k\} \in \mathbb{R}^{2k}$$

4) Re-optimize all current parameters.
5) Repeat the process until a stopping criterion is met.

The stopping criteria may include:

- A gradient-norm threshold,
- A maximum circuit depth limit,
- Or convergence based on energy difference.

Within the proposed framework, only circuits that achieve a predefined approximation ratio are retained.

$$\alpha \equiv \frac{\langle \psi(\gamma, \beta) | H_c | \psi(\gamma, \beta) \rangle}{C_{\max}}$$

where C_{max} is the maximum possible Max-Cut value of the graph and H_c is the Max-Cut cost Hamiltonian. This ratio satisfies $0 \leq \alpha \leq 1$ and $\alpha = 1$ corresponding to the optimal solution.

To increase data diversity, the initial parameters γ_o are varied, generating multiple valid quantum circuits for each graph. Afterward, only circuits that satisfy the required approximation ratio ϕ are kept.

4.5 Tokenization & Training Set Construction

4.5.1 Graph Tokens

Each graph G is encoded as a weighted edge list. The token sequence begins with a start token ⟨bos⟩ and ends with an ⟨endofgraph⟩ token:

$$\langle bos \rangle, (i_1, j_1), w_{i_1 j_1}, ..., (i_m, j_m), w_{i_m j_m}, \langle endofgraph \rangle$$

Here:

- $i_k, j_k \in 0,1, ..., n-1$ are node indices of a graph with n nodes.

- (i_k, j_k) corresponds to edges in the graph.
- $w_{ij} \in (0,1]$ is the edge weight (with unweighted graphs encoded as $w_{ij} = 1$).

All numerical values, including edge weights and parameters, are rounded to two decimal places and clipped to the range [0,1] to ensure numerical stability.

4.5.2 Circuit Tokens

Each ADAPT-QAOA circuit is encoded as a sequence of instruction blocks organized by layer. One layer is represented as:

$$\langle newlayer \rangle, O_k, \gamma_k, \beta_k$$

Where:

- $O_k \in 0, ..., m-1$ is the operator identifier from the operator pool.
- γ_k, β_k are the optimized parameters for the cost Hamiltonian and mixer at layer k.

All real-valued parameters (γ_k, β_k) are:

- Rounded to two decimal places,
- Clipped to the range $[-10,10]$

Any circuit containing parameters outside this range is discarded.

4.5.3 Instance-Isolated Sequence Construction

Each graph - circuit pair d_i forms an independent token sequence.

To preserve the structure of each instance:

- Multiple graphs or circuits are never concatenated into the same sequence.
- The model never sees tokens from different instances within the same context window, unlike standard LLM training.

To match the transformer's context length T:

- A random window size $b_i \sim Uniform(b_{\min}, b_{\max})$ with $b_{\max} \leq T$ is selected for each sequence.
- A sliding window of size b_i is used to split the sequence into subsequences.
- These subsequences are used to create supervised training pairs.

Each subsequence is right-padded to length T, and appropriate attention masks are applied during batching.

This process produces the final tokenized dataset used for training, while strictly maintaining instance boundaries, a crucial property to ensure that generated circuits have clear and reliable structure.

4.6 Model Training (Graph-Conditioned Transformer)

The tokenized graph - circuit dataset is used to train decoder-only transformer models for quantum circuit generation. The implementation is based on a modified NanoGPT / LLaMA-style causal transformer, trained from scratch without pretrained weights, and extended with graph-conditioned embeddings that provide global information about the input graph.

Two architectures are supported:

- GPT-style transformer (NanoGPT implementation)
- LLaMA-style transformer

Both models share the same input representation and training procedure.

4.6.1 Input Representation

Each token embedding combines multiple components to capture both local and global structure:

- Token embedding E_{tok} represents the discrete token in the vocabulary

- Positional embedding E_{pos} encodes the token's position in the sequence
- Broadcasted graph embedding e_G a fixed vector representing the entire graph (produced by a graph embedding model). This vector is added to all token positions to maintain global graph context

The final transformer input at each position in the sequence is:

$$X = E_{tok}(x_{1:T}) + E_{pos} + e_G$$

Here, T is the sequence length, and broadcasting ensures that every token has access to the full graph context at each decoding step.

4.6.2 Model Architecture

4.6.2.1 NanoGPT

The architecture consists of a stack of Transformer decoder blocks with causal self-attention. Each input token is first mapped to a token embedding and combined with a positional embedding. In this work, an additional graph embedding projection is added and summed with the token and position embeddings to condition the model on graph structure information.

Each Transformer block contains:

- Layer normalization
- Multi-head causal self-attention
- Residual connection
- Feed-forward MLP with GELU activation
- Dropout regularization

Causal masking ensures that each token attends only to previous tokens, enabling autoregressive generation. Padding masks are also supported so that padded tokens do not contribute to attention or loss.

The final hidden states are passed through a linear output layer tied with the token embedding matrix to produce logits over the vocabulary. Training uses cross-entropy loss with padding tokens ignored.

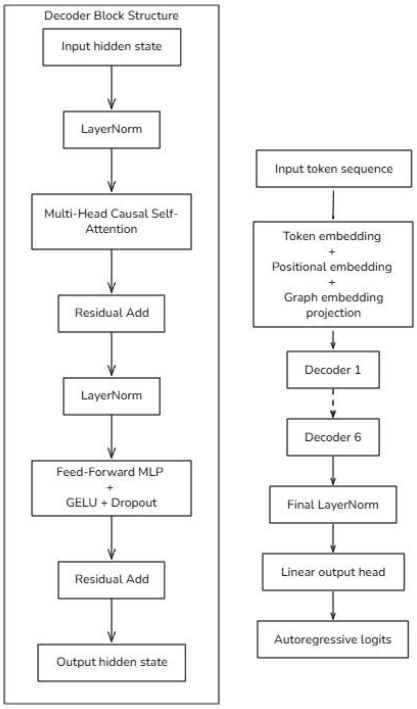

Figure 4-4 NanoGPT Architecture. Left: internal structure of a decoder block with causal self-attention, residual connections, and feed-forward layers. Right: full autoregressive pipeline incorporating token, positional, and graph embeddings.

4.6.2.2 LLaMA

The model takes token sequences as input and predicts the next token autoregressively. Similar to the GPT version, an additional graph embedding projection is added to condition the model on graph features. The projected graph embedding is added to the token embeddings before entering the Transformer blocks.

Each Transformer block consists of:

- RMSNorm normalization
- Multi-head causal self-attention with RoPE positional encoding

- Residual connection
- SwiGLU feed-forward network
- Second RMSNorm + residual connection

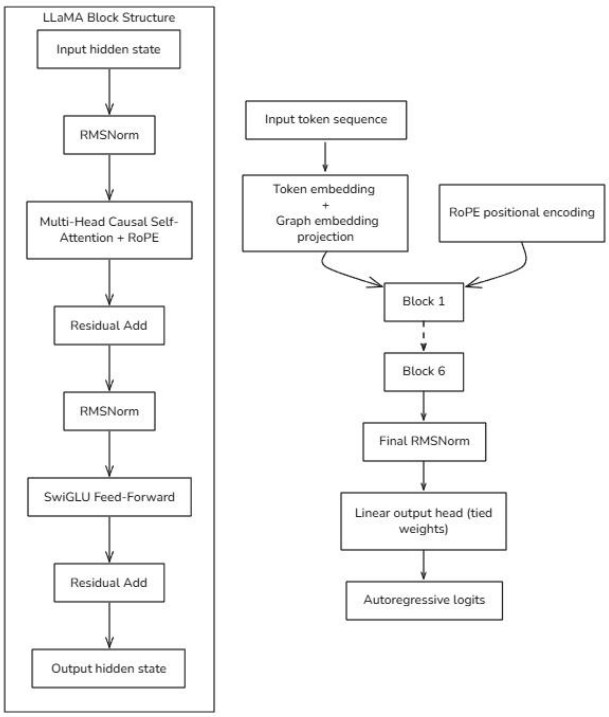

Figure 4-5 LLaMA Architecture [19]. Left: internal block structure with RMSNorm, multi-head causal self-attention with RoPE, and SwiGLU feed-forward layers. Right: full autoregressive pipeline incorporating token embeddings, graph embeddings, and rotary positional encoding.

Causal masking ensures autoregressive generation, while padding masks prevent padded tokens from affecting attention and loss computation. Unlike GPT, positional information is encoded using Rotary Positional Embeddings (RoPE) instead of learned position embeddings. Layer normalization is replaced with RMSNorm, and the standard GELU MLP is replaced with SwiGLU, following the LLaMA architecture design.

The final hidden states are passed through a linear output layer tied with the token embedding matrix to produce logits over the vocabulary. Training uses cross-entropy loss with padding tokens ignored.

CHAPTER 5. EXPERIMENTAL ENVIRONMENT

This chapter presents the experimental environment used to evaluate the proposed framework. Section 5.1 describes dataset construction for both LLM and GNN models, while Section 5.2 outlines the hardware setup and transformer configurations. Section 5.3 introduces the training objective, loss function, and evaluation metrics, including approximation ratio and circuit validity. Section 5.4 explains the training procedures for GNN and LLM models, and Section 5.5 presents the benchmarking protocol and comparison with baseline methods. The chapter also discusses the trade-off between approximation quality and circuit depth in generated quantum circuits.

5.1 Dataset

5.1.1 Training Dataset

To the best of our knowledge, there is no publicly available dataset specifically designed for training LLM-based QAOA models. Therefore, this work constructs a synthetic dataset composed of randomly generated ER graphs of varying sizes. The final dataset includes:

- 9-node graphs: 80,000 samples
- 10-node graphs: 85,887 samples
- 11-node graphs: 50,000 samples

The dataset is split into train/validation/test sets with a ratio of 8/1/1. Consequently, each model is trained solely on graphs of its target size rather than on a mixed-size dataset. Each graph is paired with a corresponding ADAPT-QAOA circuit, forming graph - circuit training instances used for supervised sequence modeling.

Due to the need to generate high-quality circuits across multiple depths and random initializations, dataset generation is computationally intensive. Depending

on graph size and circuit complexity, the process ranges from several hours to multiple days using parallel execution across two machines.

For the GNN model, a separate synthetic dataset is generated for contrastive self-supervised training of graph embeddings. The dataset consists of 500 randomly generated graphs with node sizes $n \in \{9,10,11\}$. Graph connectivity is controlled using an edge probability sampled from the range $p \in [0.2,0.9]$ allowing the dataset to capture a diverse set of graph structures. To improve robustness across different problem settings, 95% of the graphs are generated as weighted graphs, while the remaining 5% are unweighted. These graphs are used to train the GNN model to learn meaningful structural representations in the embedding space.

5.1.2 Evaluation Dataset

As no publicly available benchmark test set exists for this task or closely related work, this work constructs a separate synthetic dataset for evaluation to ensure fair and consistent comparison across methods.The evaluation set consists of 100 independently generated ER graphs for each graph size $n \in \{9,10,11\}$ resulting in a total of 300 evaluation graphs. These graphs are generated independently from the training data and are not included in any training split, ensuring a proper assessment of model generalization performance.

5.2 Experimental Setup

5.2.1 Hardware Setup

Dataset generation was performed using two local machines:

- **Laptop**: Intel Core i7-10750H CPU, NVIDIA GeForce RTX 3060 Laptop GPU (6 GB VRAM)
- **Desktop PC**: Intel Core i5-13500 CPU, NVIDIA GeForce RTX 3060 (12 GB VRAM)

Dataset generation used process-level parallelism across both machines, since the ADAPT-QAOA implementation is single-threaded and does not support multi-thread execution within a single process.

All model training experiments were conducted using CUDA acceleration on a single NVIDIA RTX 3060 GPU equipped with 12 GB of VRAM.

5.2.2 Model Setting

For both NanoGPT and LLaMA models, a unified architecture configuration is adopted to ensure consistent comparison and stable training.

The models use a context length of 256 tokens, which defines the maximum sequence size processed during training. Each model consists of 6 transformer layers with 6 attention heads, allowing sufficient capacity to capture sequential dependencies in graph-conditioned circuit tokens.

The hidden size is set to 384, providing a balance between representation power and computational cost. A dropout rate of 0.2 is applied throughout the transformer layers to reduce overfitting and improve generalization.

Overall, these settings are chosen to balance model capacity and efficiency, ensuring effective learning while remaining feasible for the available hardware and dataset scale.

5.3 Training Objective

The model is trained using an autoregressive next-token prediction loss, meaning it predicts token x_{t+1} based on the previous tokens $x_1, \ldots, x_t$. This allows the transformer to learn valid sequences of graph-conditioned quantum circuit tokens.

5.3.1 Training Loss

The training objective is based on the cross-entropy loss computed over the vocabulary, which measures the discrepancy between the predicted probability

distribution produced by the model and the true next token in the sequence. This allows the model to learn how to assign higher probabilities to correct tokens while penalizing incorrect predictions.

To enable efficient batching during training, input sequences are padded to a uniform length. However, these padding tokens do not carry meaningful information, so they are explicitly ignored when computing the loss to avoid introducing noise into the learning process.

In addition, teacher forcing is employed during training, meaning that the ground-truth token from the previous time step is always fed into the model as input when predicting the next token. This strategy helps stabilize training, reduces error accumulation across time steps, and significantly accelerates convergence.

5.3.2 Evaluation Metrics

Although the model is optimized using cross-entropy loss, additional domain-specific metrics are evaluated during training to measure the quality of generated circuits.

Approximation Ratio (AR)

Generated circuits are converted to quantum circuits and evaluated using ADAPT-QAOA.

$$AR = \frac{C}{C_{\max}}$$

Where C_{max} is the maximum possible Max-Cut value of the graph. This ratio satisfies $0 \leq AR \leq 1$ and $AR = 1$ corresponding to the optimal solution. AR is computed on held-out graphs at regular intervals during training. Higher AR indicates better solution quality

Error Rate (ER)

A generated circuit is considered valid if:

- The token sequence can be parsed
- The circuit follows the ADAPT-QAOA ansatz structure
- Energy evaluation succeeds

Error rate is the percentage of generated circuits that violate any of these conditions. This metric measures whether the model fails to learn the correct circuit syntax and structure.

Validation Loss

Cross-entropy loss on the validation set is continuously monitored to detect overfitting and to select the best training checkpoints. Training is terminated when the validation loss no longer improves, the approximation ratio (AR) reaches a performance plateau, or the circuit generation error rate begins to increase.

Numbers of Circuit Layers

Each circuit layer is represented as:

$$\langle \text{newlayer} \rangle, O_k, \gamma_k, \beta_k$$

The number of layers equals the number of ⟨newlayer⟩ tokens, or equivalently the number of parameter pairs (γ, β). A smaller number of layers generally results in faster optimization and lower computational cost, while a larger number of layers increases the expressive power of the circuit and can potentially achieve a higher approximation ratio. This thesis focuses on achieving high AR while minimizing the number of layers. Compact circuits with fewer layers are preferred, as they reduce optimization time while maintaining strong performance.

5.4 Training Procedure

5.4.1 Training Procedure for GNN

The Graph Neural Network (GNN) is trained using a fixed configuration designed to balance learning capacity and computational efficiency for small graph

structures. The training process is conducted over 20 epochs, which is sufficient for convergence given the relatively small dataset size. An Adam optimizer is used with a learning rate of 1e-3 and weight decay of 1e-4 to ensure stable optimization and prevent overfitting. Training is performed with a batch size of 32, allowing efficient utilization of GPU memory while maintaining gradient stability.

The model architecture consists of 3 layers with a hidden dimension of 128, followed by a projection to a 500-dimensional embedding space. This configuration enables the model to capture both local and multi-hop structural information while producing fixed-length graph embeddings suitable for downstream tasks.

5.4.2 Training Procedure for LLMs

The transformer-based models (NanoGPT and LLaMA) are trained using autoregressive sequence modeling with carefully selected optimization settings.

Training is performed with a batch size of 64 sequences, and each sequence has a context length of 256 tokens, which defines the maximum input size for the model. The training runs for a maximum of 30,000 iterations, although early stopping may terminate training earlier.

The optimization uses the AdamW optimizer with parameters $\beta_1 = 0.9$ and $\beta_2 = 0.95$, along with a learning rate of 1e-4. A cosine decay schedule is applied to gradually reduce the learning rate over time, preceded by 100 warm-up steps to stabilize the initial training phase. Weight decay is incorporated through AdamW regularization to improve generalization.

To reduce overfitting, a dropout rate of 0.2 is applied within transformer layers. Additionally, gradient clipping is enabled when necessary to prevent instability during training. An early stopping mechanism is used, where training terminates automatically if the validation loss does not improve for three consecutive evaluation steps.

During training, model performance is monitored at regular intervals. The training loss is evaluated every 250 iterations, while the approximation ratio is evaluated every 500 iterations on held-out data. Each evaluation step uses 200 batches, ensuring reliable estimation of model performance across the dataset.

5.5 Benchmarking Protocol

To evaluate the proposed framework, this thesis generates multiple candidate circuits for each test graph. Specifically, five circuits are produced for every graph instance, and the final performance is reported as the average Approximation Ratio (AR) across these circuits. This helps reduce randomness and provides a more stable estimate of model performance.

Each generated circuit is evaluated using several criteria. The main metric is the approximation ratio, which measures the solution quality. In addition, error rate and efficiency are also considered. Efficiency is measured by the circuit depth, such as the number of layers, which reflects the computational cost. These metrics together provide a more complete view of both performance and practicality.

To ensure a fair comparison, a fixed number of test graphs is used for each graph size. This avoids bias toward any specific graph size and allows consistent evaluation across different levels of problem complexity. All evaluation graphs are taken from the dataset described in Section 5.1.2 and are not used during training.

The proposed method is compared against two baseline approaches: Vanilla QAOA and ADAPT-QAOA. For ADAPT-QAOA, circuits are generated using the Adapt.jl package [21], which builds circuits in an adaptive manner. For Vanilla QAOA, circuits are generated using the QAOA package [20]. Since Vanilla QAOA does not adapt its structure, the number of layers is fixed. In this thesis, the depth is set to $p = n$ for all graph sizes to provide a consistent and comparable baseline.

CHAPTER 6. RESULTS & ANALYSIS

This chapter presents the experimental results and analysis of the proposed LLM-GNN assisted QAOA framework. Section 6.1 evaluates graph embedding methods, including NetLSD, FEATHER, and GNN. Section 6.2 analyzes the training behavior of the sequence models in terms of convergence, approximation ratio, error rate, and circuit depth optimization. Section 6.3 compares the proposed framework with Vanilla QAOA and ADAPT-QAOA across different graph sizes, focusing on solution quality and circuit efficiency. Finally, Section 6.4 investigates the impact of embedding methods and model architectures, including NanoGPT and LLaMA, on stability and generalization.

6.1 GNN Training Results and Embedding Analysis

6.1.1 GNN Training Results

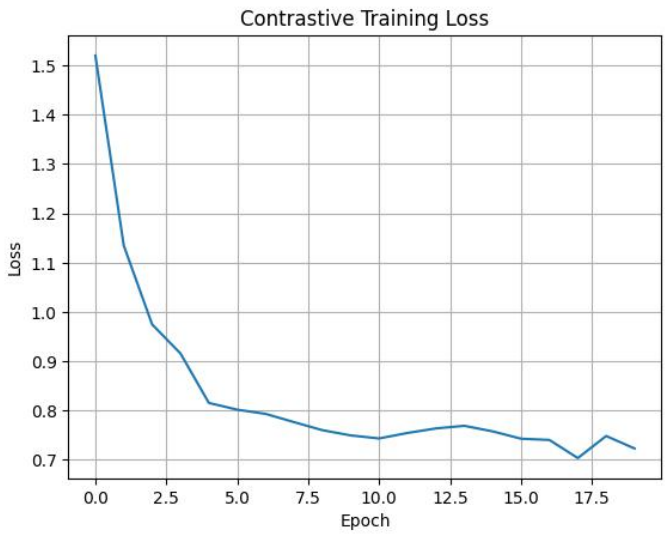

Figure 6-1 Contrastive Training Loss of GNN

As Figure 6-1, the training process of the GNN model shows clear and stable convergence behavior. The contrastive training loss decreases significantly during the early stage of training, dropping from approximately 1.5 to below 0.8 within the

first 5 epochs. After this initial rapid convergence phase, the loss stabilizes and fluctuates slightly within the range of 0.7 - 0.8 for the remaining epochs. This behavior indicates that the model quickly learns meaningful graph representations and continues to refine them without instability. No signs of divergence or exploding gradients are observed throughout training, and the encoder effectively aligns positive graph pairs while maintaining separation between different graphs.

To further validate the efficacy of the Graph Neural Network (GNN) encoder in generating meaningful embeddings, a controlled qualitative experiment was conducted to assess how structural variations translate into the embedding space. The test involved generating three distinct categories of graphs to evaluate the model's sensitivity to topological changes:

- Base Graph: A connected ER graph with $n = 10$ nodes and an edge probability of $p = 0.3$.
- Slightly Modified Graphs: 10 variations of the base graph created by removing a single edge while maintaining connectivity.
- Different Graphs: 10 independently generated ER graphs using the same parameters as the base, representing entirely different structural configurations.

After extracting high-dimensional embeddings from the GNN encoder, a distance matrix was computed using cosine distance. Multidimensional Scaling (MDS) was then employed to project these distances into a 2D plane for visualization, ensuring that the spatial relationships between the points remained representative of their similarity in the original embedding space.

Figure 6-2 provides clear evidence of the GNN's capability:

- Structural Sensitivity: The "Slightly Different" graphs (green) form a tight cluster around the "Base Graph" (blue), indicating that the encoder correctly identifies them as highly similar despite minor topological edits.

- Global Differentiation: In contrast, the "Different Graphs" (red) are widely dispersed across the embedding space, demonstrating that the model effectively captures the unique structural signatures of unrelated graphs.

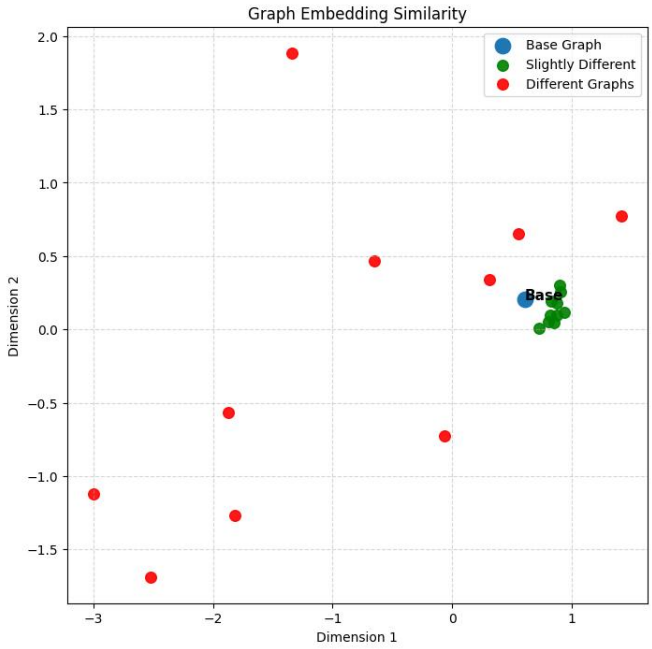

Figure 6-2 Visualization of GNN Graph Embeddings for Structural Similarity Validation.

These findings confirm that this GNN framework produces a robust embedding space where geometric proximity correlates directly with structural similarity.

6.1.2 Embedding Analysis

This section presents a comparative analysis of different graph embedding methods, including NetLSD, FEATHER, and GNN, based on their ability to capture structural information and distinguish between graph instances.

6.1.2.1 Experimental Setup

An experiment is conducted using a controlled set of randomly generated graphs. A total of 100 graphs are generated, each with 10 nodes and an edge probability of 0.4. All graphs are weighted, with edge weights sampled uniformly from the range [0.5, 2.0]. For the GNN model, node features are initialized with a

dimension of 4. To evaluate embedding stability, a perturbation test is applied by randomly modifying 10% of edges. All embedding methods produce fixed-length representations with dimension 500. The configurations for each method are as follows:

- NetLSD uses 500 scale steps, with approximations set to 50, and scale bounds defined in the range [-2, 2].
- FEATHER is configured with order 5, 25 evaluation points, a maximum theta value of 2.5, and mean pooling.
- The GNN model is implemented using a 3-layer GIN architecture with hidden dimension 128, producing 500-dim embeddings. It is trained using contrastive self-supervised learning with temperature parameter $T = 0.2$.

6.1.2.2 Pairwise Cosine Distance Analysis

The cosine distance analysis reveals clear differences in how each method separates graph representations. NetLSD produces extremely small cosine distances, indicating that most embeddings are highly similar and lack discriminative power. FEATHER shows moderate variation in distances, resulting in clearer separation between graphs. In contrast, the GNN produces the largest distance variation, demonstrating strong inter-graph separation. This suggests that contrastive training enables the GNN to capture subtle structural and edge-weight differences more effectively than the other methods.

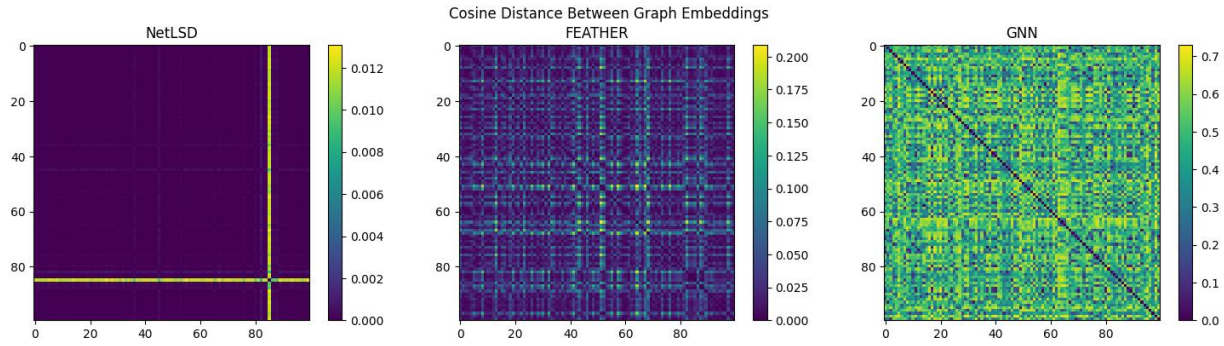

Figure 6-3 Pairwise Cosine Distance Analysis for NetLSD, FEATHER and GNN

6.1.2.3 PCA Projection Analysis

A 2D PCA projection is used to visualize the embedding space. NetLSD embeddings form a very tight cluster with low variance, showing limited ability to distinguish between graphs. FEATHER embeddings are more widely dispersed, with noticeable variation and the presence of some outliers. GNN embeddings exhibit a moderate spread with more structured grouping patterns.

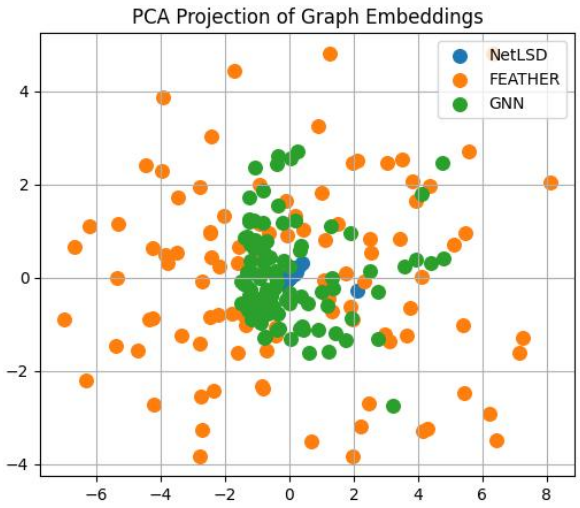

Figure 6-4 PCA Projection of Graph Embedding for NetLSD, FEATHER and GNN

Overall, the degree of separation improves from NetLSD to GNN, while FEATHER shows the highest dispersion. The GNN provides a balance between compactness and structural discrimination, avoiding both excessive similarity and uncontrolled dispersion.

6.1.2.4 Embedding Variance Analysis

Variance analysis further supports these observations. NetLSD exhibits extremely low variance, indicating that embeddings are nearly identical across different graphs. FEATHER shows the highest variance, reflecting strong sensitivity to structural differences. GNN demonstrates moderate variance, suggesting a balanced but less expressive representation compared to FEATHER. These variance patterns are consistent with the cosine distance and PCA results.

Method	Mean Per-Dimension Std
NetLSD	0.009303
GNN	0.107253
FEATHER	0.168366

Table 6-1 Mean-Per Dimension Std of NetLSD, Feather and GNN

6.1.2.5 Stability Under Edge Perturbation

To evaluate robustness, embeddings are tested under a perturbation rate of 10%. NetLSD shows minimal changes in embeddings, indicating high stability but low sensitivity to structural variations. FEATHER exhibits the largest changes, suggesting strong sensitivity to perturbations. GNN demonstrates moderate sensitivity, positioned between NetLSD and FEATHER.

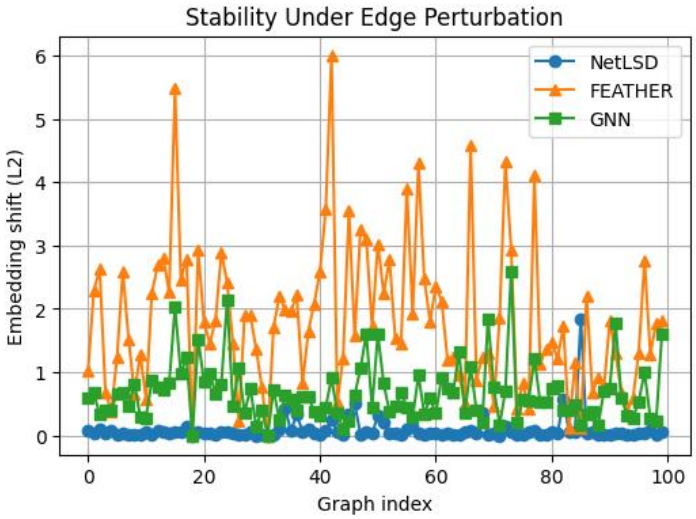

Figure 6-5 Stability Under Edge Perturbation for NetLSD, FEATHER and GNN

This highlights an important trade-off: robustness decreases as structural sensitivity increases. In this context, the GNN achieves a balanced behavior, maintaining stability while still responding meaningfully to structural changes in the graph.

Overall, the results show that NetLSD is highly stable but lacks discriminative power, FEATHER provides higher variability but can be overly sensitive, and GNN offers the most balanced performance, effectively capturing structural differences while maintaining reasonable robustness.

6.2 Training Progress and Learning Dynamics

This section analyzes the training behavior of the NanoGPT model trained on the 10-node dataset using GNN-based graph embeddings. The results are evaluated based on training and validation loss, as well as domain-specific metrics such as approximation ratio, error rate, and circuit depth.

6.2.1 Training and Validation Loss

The training process follows a typical convergence pattern. Both training loss and validation loss decrease rapidly during the early iterations and then gradually stabilize as training progresses. The validation loss reaches its minimum around iteration 3200, which corresponds to the best-performing model under the early stopping criterion. Training is automatically terminated when the validation loss does not improve for three consecutive evaluation steps.

Figure 6-6 Train Loss and Validation Loss for NanoGPT run with GNN Embedding

The close alignment between the training and validation curves indicates stable optimization and suggests that overfitting is limited in this configuration. This demonstrates that the model generalizes well to unseen validation data while maintaining consistent learning behavior.

6.2.2 Learning Dynamics

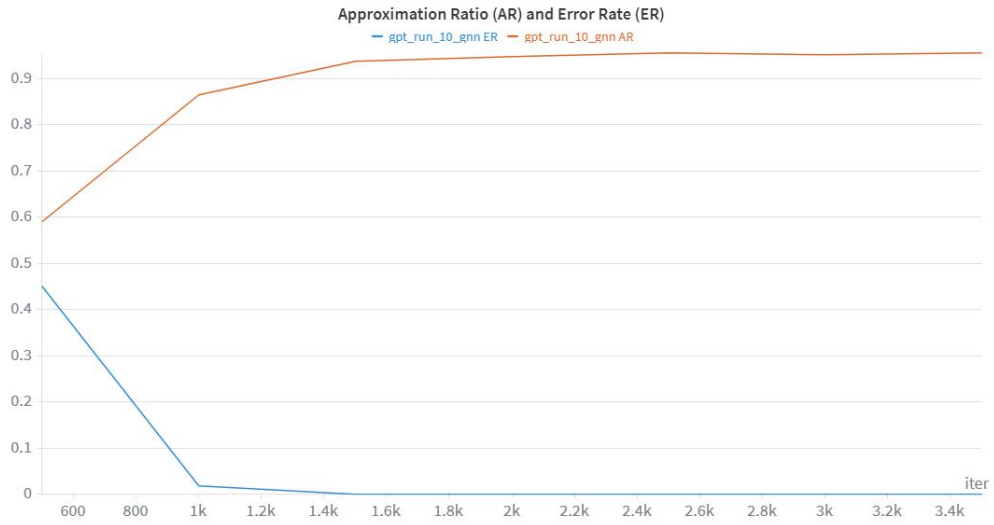

Figure 6-7 Approximation Ratio and Error Rate in NanoGPT training with GNN embedding

The learning dynamics further illustrate how the model improves during training. Rapid convergence is observed in the early stages, where significant performance gains occur within the first 1,500 iterations. During this phase, the approximation ratio (AR) increases sharply and exceeds 0.9, while the error rate (ER) falls below 0.1 within the first 1,000 iterations. The model performance then gradually stabilizes, with the error rate reaching 0 at approximately 2,000 iterations, indicating that the model consistently generates valid circuit structures.

Early acquisition of high-quality patterns: The approximation ratio remains relatively stable after the initial phase, suggesting that the model quickly learns to generate high-quality circuits early in training. This behavior is expected, as the training dataset consists only of circuits with high approximation ratios.

6.2.3 Circuit Depth Optimization

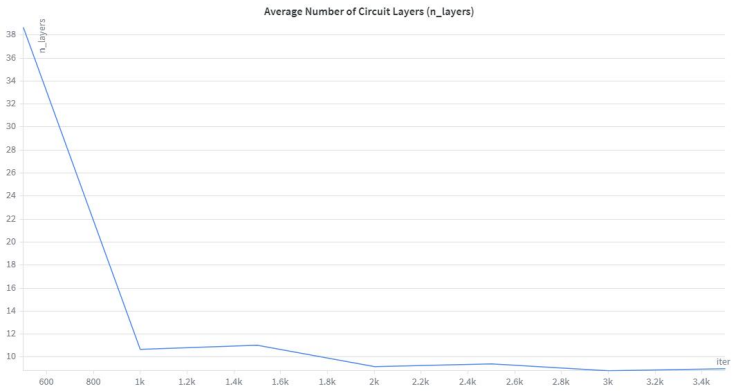

Figure 6-8 Average Number of Circuit Layers for NanoGPT training with GNN Embedding

The evolution of circuit depth, measured by the average number of layers, shows a clear optimization trend:

- The training process initially starts with circuits containing up to 30 layers.
- Around iteration 1000, a sharp reduction occurs, bringing the average number of layers down to approximately 11.
- After 2000 iterations, the layer count stabilizes and remains consistently below 10.

This trend indicates that the model learns to generate more compact circuits while maintaining high performance. The reduction in circuit depth reflects improved efficiency, as fewer layers lead to lower optimization cost without sacrificing solution quality.

6.2.4 Approximation Ratio Across Model Architectures

This experiment compares the training behavior of two transformer architectures, NanoGPT and LLaMA, using GNN-based graph embeddings on the 10-node dataset. The approximation ratio (AR) is monitored across training iterations to evaluate how quickly and effectively each model learns to generate high-quality quantum circuits.

In the early stage of training, NanoGPT demonstrates very rapid convergence, with the AR increasing sharply from approximately 0.6 to above 0.9 within the first 1,000 iterations. After this point, the model quickly stabilizes and maintains a consistently high AR close to optimal performance. This indicates that NanoGPT is highly efficient at leveraging GNN embeddings to learn the mapping between graph structure and circuit design.

In contrast, LLaMA exhibits a more gradual learning curve. The AR improves steadily over time, starting from a lower initial value and requiring significantly more iterations to approach the performance level of NanoGPT. While LLaMA eventually reaches a competitive AR, its convergence is slower and shows minor fluctuations during later stages of training.

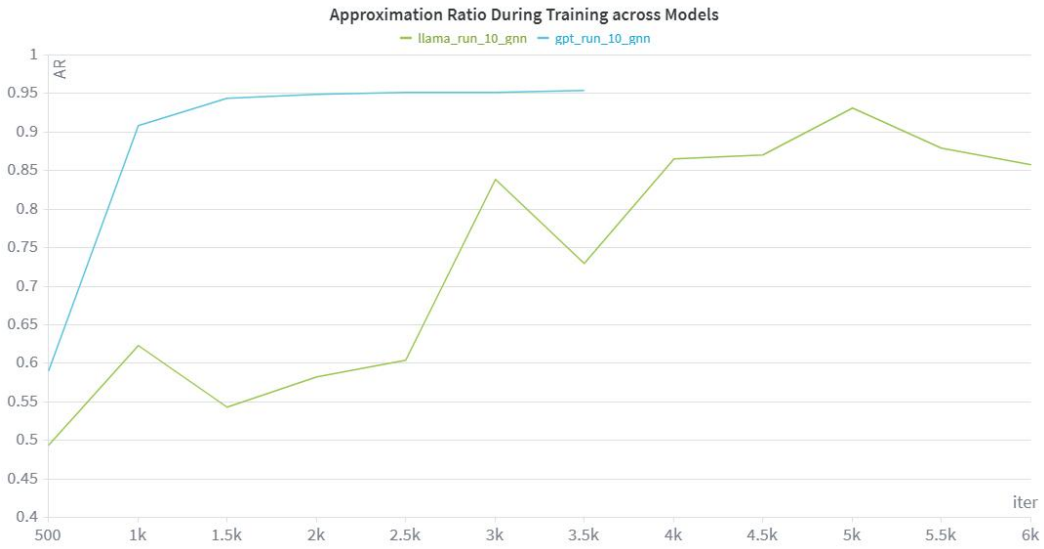

Figure 6-9 Approximation Ratio During Training Across Models (NanoGPT vs LLaMA with GNN embeddings)

Overall, the results suggest that NanoGPT achieves faster convergence and reaches stable performance earlier during training, whereas LLaMA demonstrates a smoother but slower improvement process. Despite these differences in learning dynamics, both models are capable of achieving high approximation ratios (AR).

6.2.5 Approximation Ratio Across Embedding Types

This experiment evaluates the impact of different graph embedding methods: GNN, NetLSD, and FEATHER - on the training performance of the NanoGPT model. The approximation ratio is tracked over training iterations to assess how effectively each embedding supports circuit generation.

All three embedding methods show rapid improvement in AR during the early training phase, with values increasing significantly within the first 1,000 iterations. This indicates that the model can quickly extract useful structural information regardless of the embedding type.

However, differences become more apparent as training progresses. The FEATHER embedding consistently achieves the highest AR, reaching near-optimal performance slightly earlier and maintaining stable results throughout training. This suggests that FEATHER provides more expressive and task-relevant structural features.

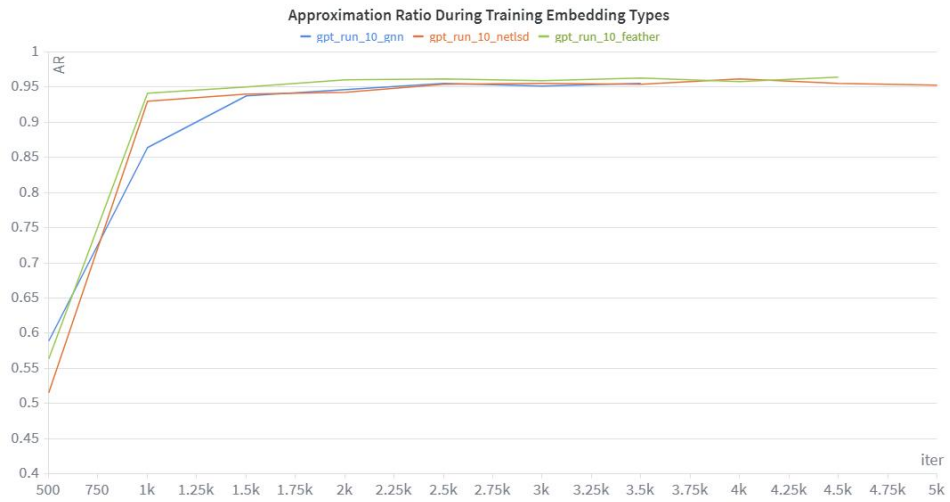

Figure 6-10 Approximation Ratio During Training Across Embedding Types (NanoGPT with GNN, NetLSD, and FEATHER embeddings)

Both NetLSD and GNN embeddings also achieve high AR values, but their performance tends to plateau slightly below that of FEATHER. While the gap is not large, these methods show less stability and slightly lower final performance,

indicating that fixed structural descriptors may be less effective than learned representations in this task.

6.3 Comparison of Vanilla QAOA, ADAPT-QAOA, and Hybrid LLM-GNN Assisted ADAPT-QAOA

This section evaluates the proposed framework on graph instances with **n = 9, 10, and 11 nodes**, using 100 ER graphs for each problem size. Edge weights are sampled from a uniform distribution $U(0,1)$. For every graph, five circuits are generated, and results are reported on multiple metrics. The comparison includes the baseline approaches as mentioned in section 5.5

6.3.1 Approximation Ratio Analysis

Nodes	Adapt mean	Qaoa mean	LLaMA -Feather	LLaMA -GNN	LLaMA-NetLSD	NanoGPT-Feather	NanoGPT -GNN	NanoGPT -NetLSD
9	**0.973**	0.861	0.941	0.927	0.903	0.955	0.951	<u>0.957</u>
10	**0.973**	0.868	0.928	0.839	0.941	<u>0.962</u>	0.954	0.96
11	**0.971**	0.874	0.913	0.924	0.93	0.961	0.95	<u>0.968</u>

Table 6-2 Approximation Ratio (AR) for Vanilla QAOA, ADAPT-QAOA and This work's methods. Results Are Presented as Mean Values Across 100 Graphs For Each n nodes graph. The best and second-best values in each row are highlighted in **bold** and <u>underlined</u>, respectively. Methods proposed in this work are additionally indicated with a green background.

Table 6-2 demonstrates that this work maintains consistently strong approximation performance across graph sizes from 9 to 11 nodes, with clear differences emerging between model families. As illustrated in Figure 6-11 to Figure 6-13, where the plotted values are directly derived from Table 6-2, the proposed approach significantly outperforms Vanilla QAOA across all settings. Specifically, Vanilla QAOA achieves mean approximation ratios in the range of 0.861 to 0.874, while this work consistently produces substantially higher values

across all node sizes. At the same time, the best-performing configurations in this work closely approach the performance of ADAPT-QAOA, whose mean AR lies around 0.971 - 0.973.

A closer inspection of the results shows that NanoGPT-based models provide the most stable and competitive performance. In Figure 6-11 to Figure 6-13, NanoGPT-Feather consistently appears near the top among the proposed methods, achieving AR values around 0.955 - 0.962 across 9, 10, and 11 nodes. Similarly, NanoGPT-GNN and NanoGPT-NetLSD remain highly competitive, generally staying within the range of 0.95 to 0.968. Importantly, their performance remains robust as the problem size increases, indicating good scalability across different graph sizes.

In contrast, LLaMA-based models exhibit greater variability, which is also evident in the visual trends of the figures. LLaMA-NetLSD performs best within this group, reaching approximately 0.903 to 0.941, while LLaMA-Feather achieves moderate results in the range of 0.913 - 0.941. LLaMA-GNN, however, shows less stable behavior, with a noticeable drop to around 0.839 at 10 nodes as shown in Figure 6-12, before partially recovering at 11 nodes in Figure 6-13. This inconsistency suggests limitations in effectively leveraging certain embedding types within this architecture.

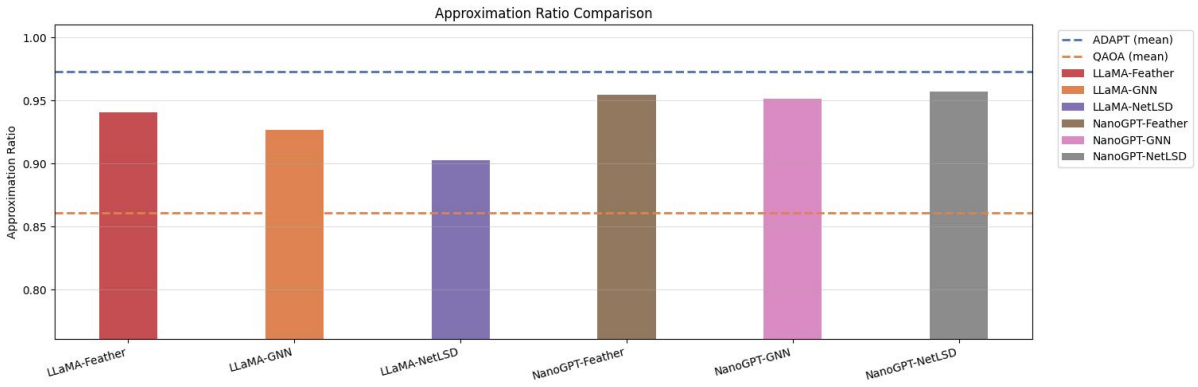

Figure 6-11 Approximation Ratio comparison for 9-node graphs

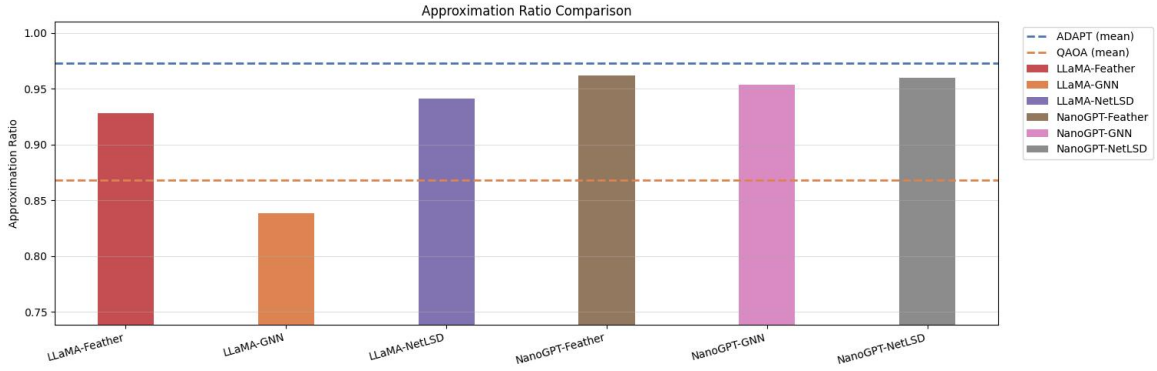

Figure 6-12 Approximation Ratio comparison for 10-node graphs

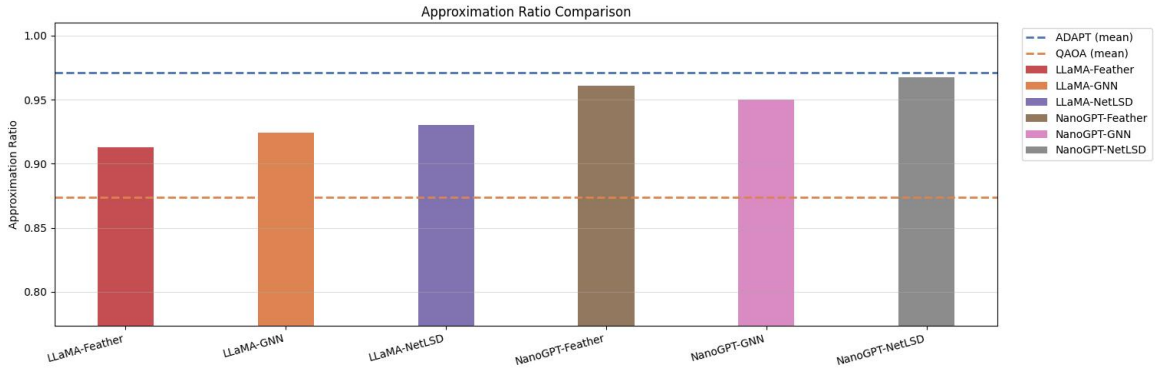

Figure 6-13 Approximation Ratio comparison for 11-node graphs

Overall, the results indicate that this work can effectively approximate high-quality ADAPT-QAOA solutions across multiple graph sizes, with NanoGPT-based variants providing the most reliable performance.

6.3.2 Circuit Depth Analysis

Turning to circuit efficiency, Table 6-3 and Figure 6-14 to Figure 6-16 present the average number of circuit layers for all methods. These figures, which again reflect the tabulated values, reveal a clear trade-off between approximation quality and circuit depth. Despite this trade-off, this work demonstrates that high-quality solutions can be achieved with significantly reduced circuit complexity.

Across all node sizes, NanoGPT-based models produce the most compact circuits. As seen in Figure 6-14 to Figure 6-16, NanoGPT-Feather consistently requires only about 7.952 to 8.908 layers, while NanoGPT-GNN typically remains within approximately 8.244 to 9.264 layers, and NanoGPT-NetLSD ranges from

about 8.850 to 11.850 layers. These values are not only lower than Vanilla QAOA, which uses a fixed depth equal to the number of nodes, but also substantially below ADAPT-QAOA, whose depth increases from about 13.138 layers at 9 nodes to 15.407 layers at 11 nodes.

Number of Circuit Layers								
Nodes	**Adapt mean**	**Qaoa mean**	**LLaMA-Feather**	**LLaMA-GNN**	**LLaMA-NetLSD**	**NanoGPT-Feather**	**NanoGPT-GNN**	**NanoGPT-NetLSD**
9	13.138	9	<u>8.806</u>	9.858	9.598	**7.952**	9.264	8.85
10	13.738	10	11.92	13.716	10.832	<u>8.688</u>	**8.638**	9.072
11	15.407	11	14.896	15.444	12.412	<u>8.908</u>	**8.244**	11.85

Table 6-3 Number of Circuit Layers for Vanilla QAOA, ADAPT-QAOA and This work's methods. Results Are Presented as Mean Values Across 100 Graphs For Each n nodes graph. The best and second-best values in each row are highlighted in **bold** and <u>underlined</u>, respectively. Methods proposed in this work are additionally indicated with a green background.

In comparison, LLaMA-based models generally require deeper circuits to achieve their results. The layer counts vary more widely, with LLaMA-Feather ranging from approximately 8.8 to 14.9 layers, LLaMA-NetLSD from about 9.6 to 12.4 layers, and LLaMA-GNN ranging from around 9.858 to 15.444 layers. In some cases, as reflected in the figures, LLaMA-GNN even exceeds the depth of ADAPT-QAOA (e.g., 15.44 vs. 15.41 at 11 nodes), highlighting inefficiencies in circuit construction.

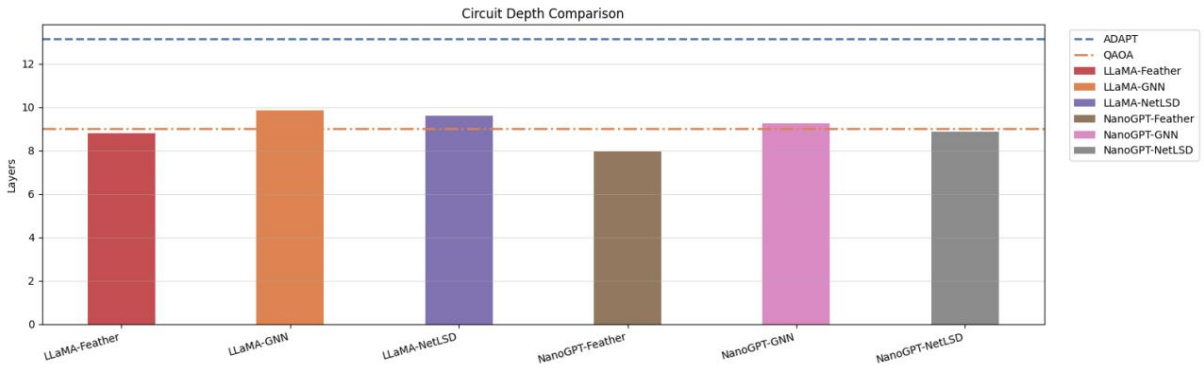

Figure 6-14 Number of Circuit Layers comparison for 9-node graphs

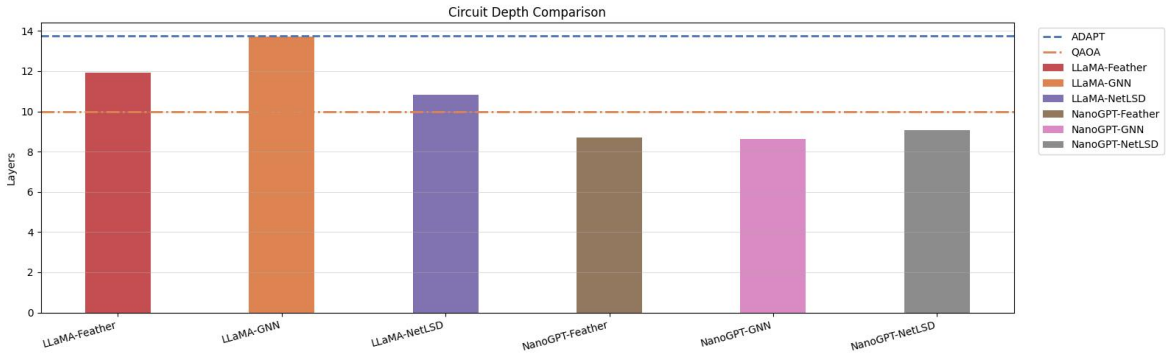

Figure 6-15 Number of Circuit Layers comparison for 10-node graphs

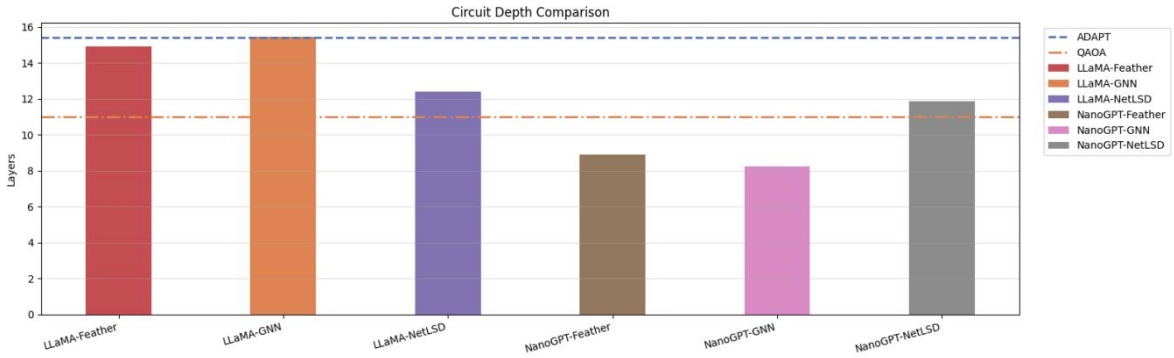

Figure 6-16 Number of Circuit Layers comparison for 11-node graphs

6.3.3 Error Rate Analysis

Figure 6-17 to Figure 6-19 present the error rates across graph sizes of 9, 10, and 11 nodes, with the corresponding numerical values summarized in Table 6-4. Since the figures are directly derived from the table, they clearly illustrate the consistency and reliability of each model across different problem sizes.

Error Rate						
Nodes	**LLaMA-Feather**	**LLaMA-GNN**	**LLaMA-NetLSD**	**NanoGPT-Feather**	**NanoGPT-GNN**	**NanoGPT-NetLSD**
9	0	0	0	0	0	0
10	0.086	0	0.016	0	0	0
11	0	0	0	0	0	0

Table 6-4 Error Rate for This work's methods. Results Are Presented as Mean Values Across 100 Graphs For Each n nodes graph

The results show that this work achieves extremely low error rates across most configurations, with several models reaching exact zero error. In particular, NanoGPT-based variants demonstrate highly stable behavior. As observed in Figure 6-17 to Figure 6-19, NanoGPT-Feather, NanoGPT-GNN, and NanoGPT-NetLSD consistently maintain zero error rates across all node sizes (9, 10, and 11 nodes). This level of consistency indicates that the smaller GPT-style architectures are highly effective at capturing the underlying structure required for accurate parameter prediction in quantum circuits.

In contrast, LLaMA-based models exhibit greater variability, which is also visible in the figures. At 9 nodes, all models achieve zero error; however, differences begin to emerge as the problem size increases. In Figure 6-18 (n = 10), LLaMA-Feather shows a noticeable increase in error rate, reaching approximately 0.086, which is the highest among all evaluated models. LLaMA-NetLSD performs better but still maintains a non-negligible error rate of around 0.016. Interestingly, LLaMA-GNN records zero error at this node size, as shown in the figure, but this does not correspond to strong approximation performance observed earlier, suggesting that the model may converge to consistent yet suboptimal solutions.

At 11 nodes, as illustrated in Figure 6-19, all models return to zero error, indicating fully stable behavior at this scale. This overall pattern reinforces the stability advantage of NanoGPT architectures while also showing that LLaMA-based models can recover stability at larger graph sizes despite earlier inconsistencies.

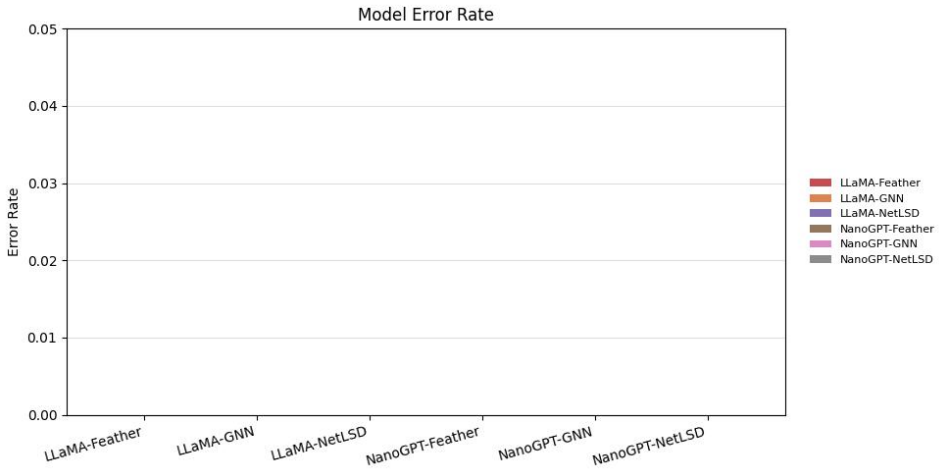

Figure 6-17 Error Rate comparison for 9-node graphs

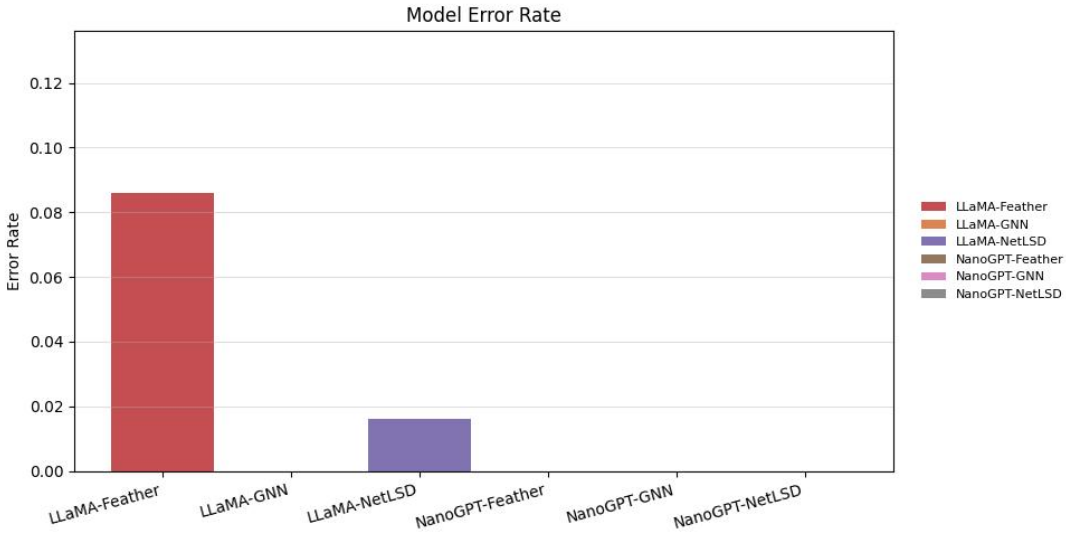

Figure 6-18 Error Rate comparison for 10-node graphs

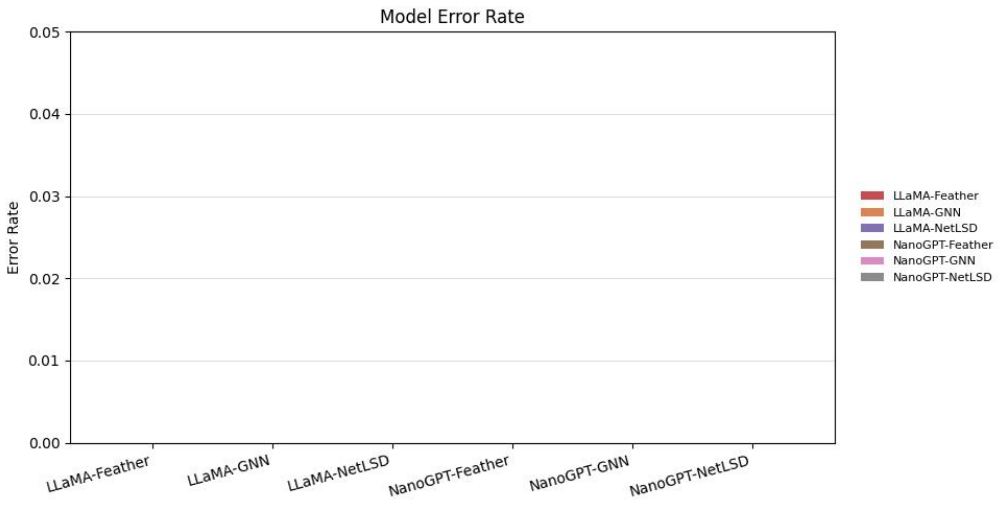

Figure 6-19 Error Rate comparison for 11-node graphs

6.3.4 AR & Layers Trade-off

The trade-off analysis highlights the relationship between solution quality, measured by the Approximation Ratio (AR), and circuit complexity, represented by the number of layers. As illustrated in the corresponding scatter plots, where each point reflects the values reported in the tables, an ideal model is expected to lie toward the upper-left region, achieving high AR while maintaining a low circuit depth.

From this perspective, the results show that this work achieves a strong balance between these two objectives. In particular, NanoGPT-Feather models form a tight cluster within the 8 to 10-layer range while consistently maintaining AR values above 0.95. This positioning clearly places them in the desired region of the plot, indicating higher efficiency compared to Vanilla QAOA, which operates at a fixed depth of 10 layers but with noticeably lower approximation quality.

The robustness of the NanoGPT framework is further supported by the behavior of NanoGPT-GNN and NanoGPT-NetLSD. As observed in the scatter plots, these models also concentrate in regions of relatively low circuit depth while preserving high AR values. This consistency across different embedding methods suggests that NanoGPT-based approaches are capable of learning efficient circuit representations without being overly sensitive to the choice of graph features.

In contrast, LLaMA-based models display significantly greater variability, which is evident from the wider spread of points toward higher layer counts. Many configurations of LLaMA-GNN and LLaMA-Feather extend into regions requiring 15 to 25 layers. Moreover, as the circuit depth increases for these models, the corresponding AR values do not consistently improve. In several cases, the AR drops substantially, reaching values as low as 0.5 to 0.7. This pattern indicates that increasing circuit complexity in LLaMA-based approaches does not necessarily lead to better performance and can, in fact, result in degradation.

When compared to ADAPT-QAOA, which appears in the plots with the highest AR values (approximately 0.98 to 1.0), a clear trade-off emerges. While ADAPT-QAOA achieves superior solution quality, it does so at the cost of significantly deeper and more variable circuit depths, sometimes reaching up to 25 layers. In contrast, this work - particularly the NanoGPT-based models, achieves near-ADAPT performance while maintaining a much more compact and consistent circuit structure, generally staying below 12 layers. This demonstrates that high-quality solutions can be obtained without the need for extensive iterative circuit construction.

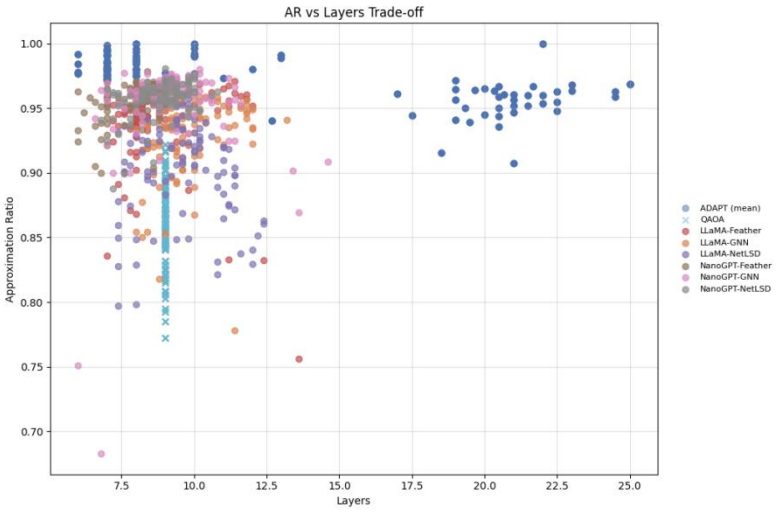

Figure 6-20 AR & Layers Trade-off for 9-node graphs

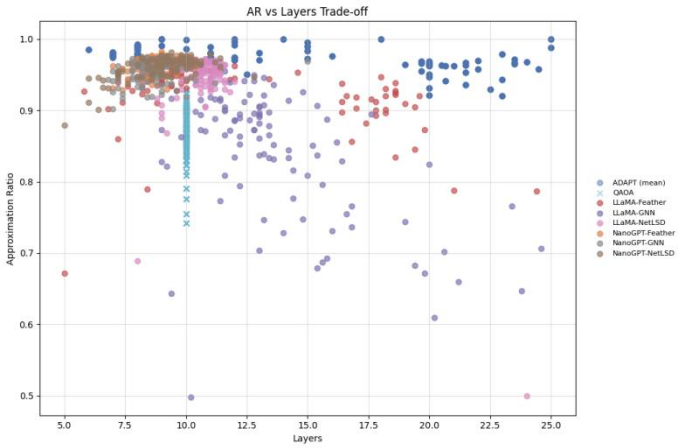

Figure 6-21 AR & Layers Trade-off for 10-node graphs

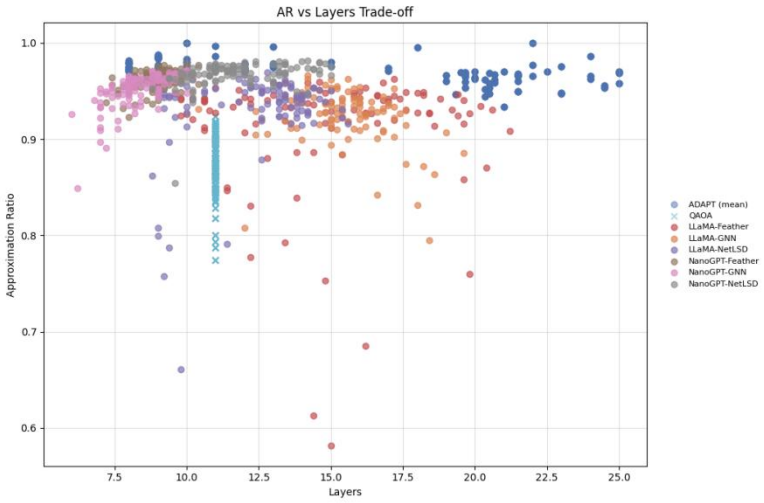

Figure 6-22 AR & Layers Trade-off for 11-node graphs

6.3.5 Summary

This section evaluates the proposed Hybrid LLM-GNN framework Assisted ADAPT-QAOA on graph instances with 9 to 11 nodes and demonstrates that this work consistently achieves strong performance across all considered metrics. By leveraging learned circuit construction instead of iterative optimization, the framework is able to closely approximate the behavior of ADAPT-QAOA while significantly reducing computational complexity.

In terms of solution quality, this work clearly outperforms Vanilla QAOA across all graph sizes, as shown in Figure 6-11 to Figure 6-13 and Table 6-2. The approximation ratios produced by the proposed models are consistently higher, with the best configurations approaching the performance of ADAPT-QAOA. Among all variants, NanoGPT-based models show the most stable and competitive results, maintaining high AR values across increasing problem sizes. In contrast, LLaMA-based models exhibit greater variability, with performance depending more strongly on the embedding type and graph size.

Regarding circuit efficiency, the results in Figure 6-14 to Figure 6-16 and Table 6-3 confirm that this work can generate significantly more compact circuits. NanoGPT-based models consistently require fewer layers than both Vanilla QAOA

and ADAPT-QAOA, while still preserving high approximation quality. On the other hand, LLaMA-based models tend to require deeper circuits and show less consistent behavior, in some cases even exceeding the depth of ADAPT-QAOA without corresponding improvements in performance.

The error rate analysis in Figure 6-17 to Figure 6-19 and Table 6-4 further highlights the reliability of the proposed approach. NanoGPT models achieve near-zero error rates across all node sizes, demonstrating highly stable parameter prediction. In contrast, LLaMA variants show occasional spikes in error, particularly at larger graph sizes, indicating less consistent generalization.

Finally, the trade-off analysis between approximation ratio and circuit depth shows that this work achieves an effective balance between solution quality and efficiency. The best-performing models are concentrated in the desirable region of high AR and low circuit depth, while avoiding the excessive complexity associated with ADAPT-QAOA.

6.4 Comparison of Different Models & Embedding Types

This section evaluates the proposed framework on graph instances with **n = 10 nodes**, using 100 ER graphs for each problem size. Edge weights are sampled from a uniform distribution $U(0,1)$. For every graph, five circuits are generated, and results are reported using average approximation ratios (AR). The comparison includes the baseline approaches as mentioned in section 5.5

6.4.1 Embedding Types Analysis

This section focuses on comparing the impact of different embedding methods - NetLSD, FEATHER, and GNN - on the performance of the proposed framework. Unlike ADAPT-QAOA, which serves as a separate baseline and does not rely on embeddings, the results here reflect how different graph representations influence the learned model's ability to generate high-quality quantum circuits.

Model Type	Model Name	Embedding Method	Adapt AR Mean	Adapt Layers	Model AR	Model Error Rate	Model Layers
LLaMA	LLaMA-Feather	Feather	0.973	13.738	0.928	0.086	11.920
	LLaMA-GNN	GNN	0.973	13.738	0.838	<u>0.002</u>	13.988
	LLaMA-NetLSD	NetLSD	0.973	13.738	0.941	0.016	10.832
NanoGPT	NanoGPT-Feather	Feather	0.973	13.738	**0.962**	**0.000**	<u>8.688</u>
	NanoGPT-GNN	GNN	0.973	13.738	0.954	**0.000**	**8.638**
	NanoGPT-NetLSD	NetLSD	0.973	13.738	<u>0.960</u>	**0.000**	9.072

Table 6-5 Performance Comparison of Embedding Methods Across LLaMA and NanoGPT Architectures. Results Are Presented as Mean Values across 100 Graphs for 10-node graph. The best and second-best values in each column are highlighted in **bold** and <u>underlined</u>, respectively.

From Table 6-5, clear differences can be observed across embedding types in terms of approximation ratio (AR), error rate, and circuit depth. Overall, NetLSD and FEATHER tend to achieve the highest AR values, while GNN shows more variability depending on the model. NetLSD consistently delivers strong and stable performance, reaching AR values up to 0.960 - 0.962 in NanoGPT and 0.941 in LLaMA. FEATHER performs similarly well in NanoGPT (0.962) but drops in LLaMA (0.928) and introduces a higher error rate.

GNN, while remaining competitive in NanoGPT with an AR of 0.9538 and zero error, performs significantly worse in LLaMA, where the AR decreases to 0.8377, despite maintaining a very low error rate (0.002). This suggests that although LLaMA-GNN produces consistent outputs, it may converge to suboptimal solutions

These differences become more apparent when examining the per-graph AR distributions shown in Figure 6-23. Across the three embedding methods, the overall AR trends remain relatively similar within the same model architecture. In NanoGPT, FEATHER, GNN, and NetLSD all maintain consistently high AR values across nearly all graph instances, with only small fluctuations and very few significant drops. The distributions are tightly clustered near the ADAPT-QAOA reference line, indicating that changing the embedding method does not substantially alter the final optimization quality for NanoGPT.

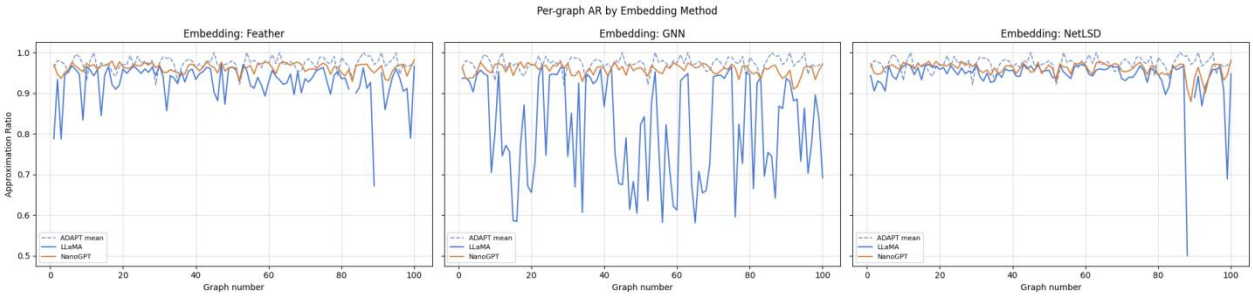

Figure 6-23 Per Graph AR by Embedding Methods on 10-node graphs

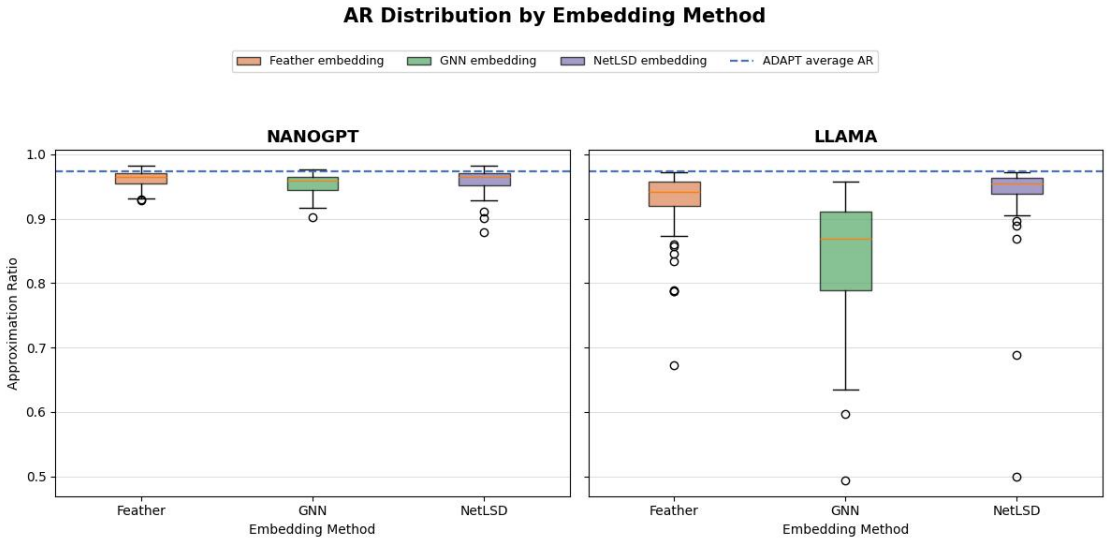

Figure 6-24 AR Distribution by Embedding Methods on 10-node graphs on model NanoGPT (left) and model LLaMA (right)

A similar trend can be observed in the AR distribution plots in Figure 6-24. In NanoGPT, all embedding methods produce tightly concentrated distributions with very similar mean AR values and almost no extreme outliers, indicating that the

model is largely insensitive to the choice of graph embedding. In contrast, the LLaMA architecture exhibits much wider AR distributions, particularly for GNN embeddings, which contain many low-value outliers. FEATHER and NetLSD also show greater fluctuations compared with their NanoGPT counterparts.

The circuit layer distributions in Figure 6-25 further support this conclusion. In NanoGPT, all embedding methods produce compact and highly similar layer distributions, with average circuit depths remaining around 8-9 layers, indicating stable circuit complexity regardless of the embedding representation. In contrast, LLaMA shows much broader layer distributions and more extreme outliers across all embeddings. Although minor differences exist between NetLSD, FEATHER, and GNN, the variation introduced by the architecture is substantially larger.

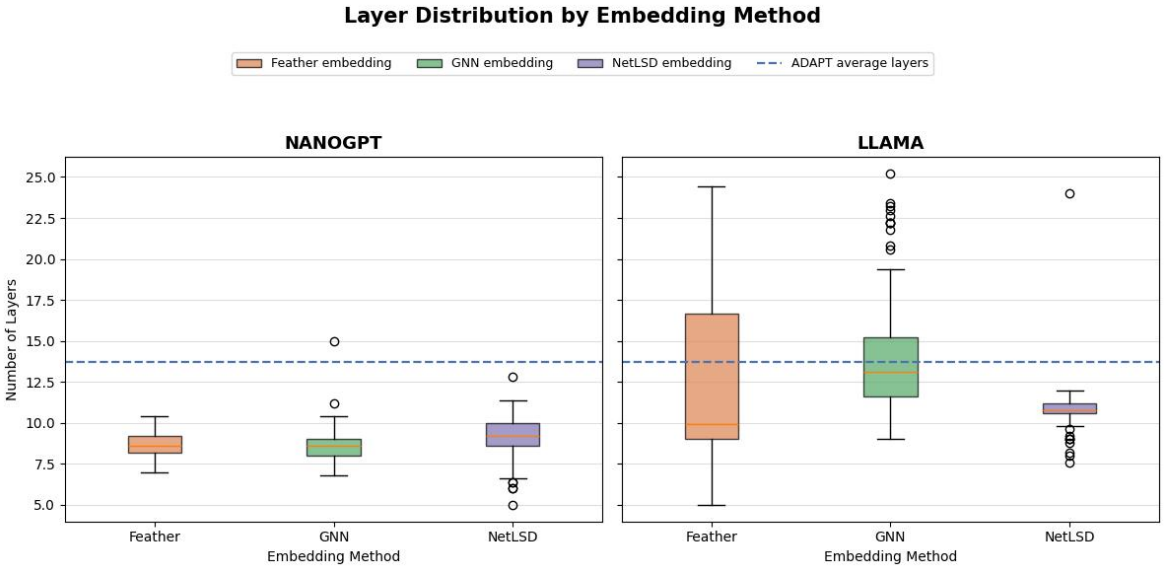

Figure 6-25 Number of Circuit Layers Distribution by Embedding Methods: NetLSD, FEATHER and GNN on 10-node graphs on model NanoGPT (left) and model LLaMA (right)

The error rate analysis in Figure 6-26 supports the same interpretation. NanoGPT achieves near-zero error rates across all embedding methods, demonstrating highly stable behavior independent of the graph representation used. In LLaMA, the error rates increase noticeably, especially for FEATHER and NetLSD, while GNN remains relatively low despite its weaker AR performance.

The contrast between NanoGPT and LLaMA is substantially larger than the differences observed among embeddings within the same architecture.

Overall, the experimental results suggest that the choice of embedding method has only a limited impact on performance compared with the influence of model architecture. While minor differences exist between FEATHER, GNN, and NetLSD, particularly in stability and circuit depth, these variations remain relatively small within the same model. Instead, the major performance gap is observed between NanoGPT and LLaMA, indicating that the architecture itself plays the dominant role in determining approximation quality, error rate, and circuit efficiency.

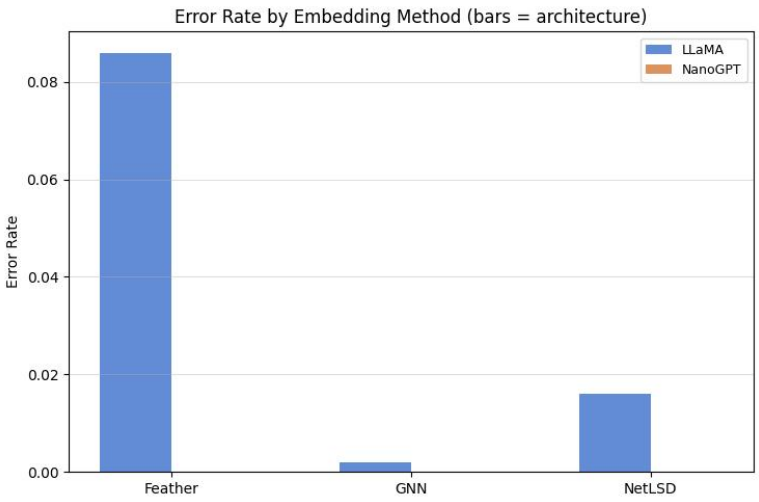

Figure 6-26 Error Rate by Embedding Method Netlsd, FEATHER and GNN on 10-node graphs

6.4.2 Model Architecture Analysis

This section analyzes the impact of model architectures, LLaMA and NanoGPT - on the performance of the proposed framework, independent of embedding choice. By aggregating results across all embedding methods, we evaluate how each architecture influences approximation quality, stability, and circuit efficiency.

From Table 6-5, NanoGPT consistently outperforms LLaMA across all embedding types in terms of approximation ratio (AR) and error rate. NanoGPT

achieves high and stable AR values ranging from 0.954 to 0.962, with zero error across all embeddings. In contrast, LLaMA shows greater variability, with AR values ranging from 0.838 to 0.941 and a noticeable error rate in the FEATHER setting (0.086), along with smaller but non-zero errors in other configurations (0.002 for GNN and 0.016 for NetLSD). This indicates that NanoGPT is more reliable and robust, while LLaMA is more sensitive to both embedding choice and graph structure.

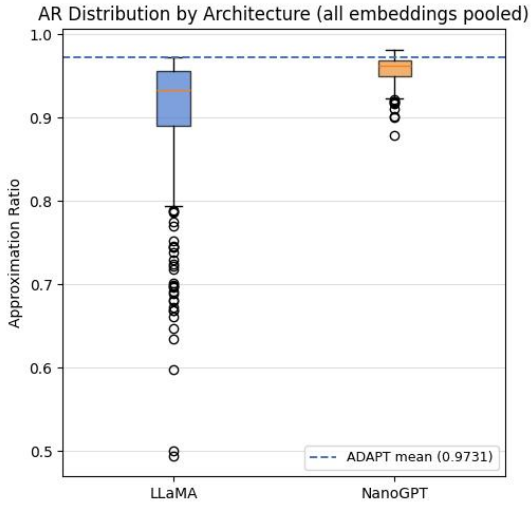

Figure 6-27 AR Distribution by Architecture 10-node graphs

The AR distribution plot (Figure 6-27) further reinforces this observation. NanoGPT exhibits a tightly concentrated distribution near the top AR range, with very few outliers and minimal spread. This suggests consistent performance across different graph instances. On the other hand, LLaMA shows a much wider distribution, with a long lower tail and many low-performing outliers, including extreme cases dropping significantly below 0.5. This highlights weaker generalization and higher sensitivity to difficult instances.

The difference in efficiency is even more pronounced when examining circuit depth. As shown in Table 6-5, NanoGPT consistently produces shallower circuits, with layer counts around 8.7 - 9.1 across all embeddings. In contrast, LLaMA requires deeper circuits, ranging from approximately 10.8 to 14 layers. This gap is

clearly visualized in the layer distribution plot (Figure 6-28), where NanoGPT has a tight and low-centered distribution, while LLaMA shows a much broader spread with many high-layer outliers, some exceeding 25 layers. Notably, NanoGPT consistently operates well below the ADAPT-QAOA baseline in terms of circuit depth, whereas LLaMA often approaches or exceeds it, indicating less efficient optimization.

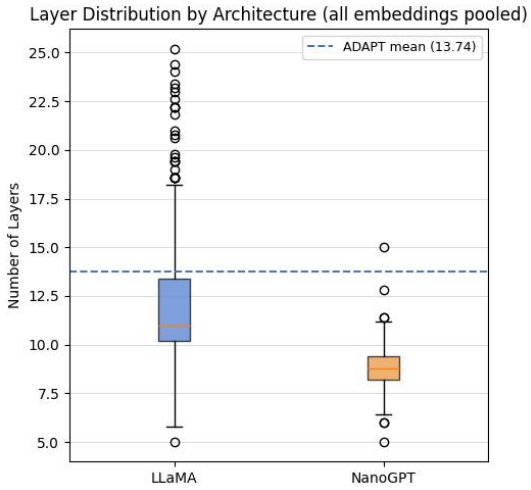

Figure 6-28 Number of Circuit Layers Distribution by Architecture 10-node graphs

These differences can be better understood by considering the architectural design and training configuration. Both models share the same capacity - 6 layers, 6 attention heads, hidden size 384, and context length 256 - ensuring that performance differences are not due to scale but to architectural choices.

NanoGPT uses a standard Transformer decoder with LayerNorm, learned positional embeddings, and a GELU-based feed-forward network. This simpler and more stable design appears well-suited for the structured and relatively short sequence generation task in this work. The use of learned positional embeddings may provide more flexibility in encoding token order specific to circuit construction, while GELU activations contribute to smoother optimization behavior. As a result, NanoGPT demonstrates strong generalization and consistent performance across graph instances and embedding types.

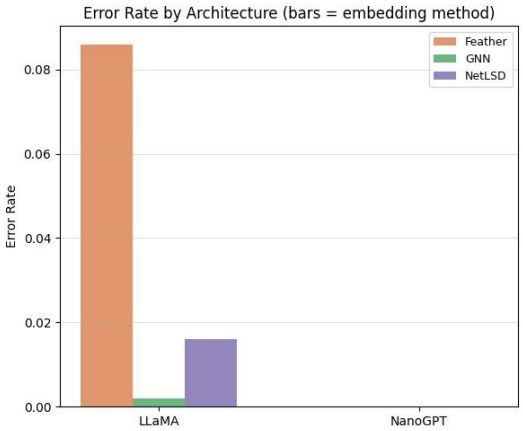

Figure 6-29 Error Rate by Architecture 10-node graphs

In contrast, LLaMA introduces several architectural modifications, including RMSNorm, Rotary Positional Embeddings (RoPE), and a SwiGLU feed-forward network. While these components are designed to improve scalability and performance in large-scale language modeling, they may introduce additional complexity in this setting. In particular, RoPE encodes positional information in a more rigid, rotational manner, which may be less aligned with the discrete and structured nature of circuit token sequences. Similarly, the higher sensitivity of SwiGLU and the absence of standard LayerNorm may lead to less stable optimization when combined with graph-conditioned inputs.

Another important factor is how graph embeddings are integrated. In both architectures, the graph embedding is projected and added to token embeddings. However, NanoGPT appears to utilize this information more effectively, maintaining stable performance regardless of embedding type. LLaMA, on the other hand, shows stronger dependence on embedding characteristics, leading to larger performance gaps across different embeddings. This suggests that NanoGPT provides a smoother interaction between sequence modeling and graph conditioning, while LLaMA may amplify variations in the input representation.

6.5 Generalization Performance Across Graph Sizes

This section evaluates the scalability and generalization capability of the proposed hybrid LLM-GNN framework across different graph sizes. It is important to emphasize that all framework variants are models trained specifically on 11-node graphs, and their performance is assessed here in terms of generalization to unseen graph sizes. Graph instances are generated for sizes 4 to 13 nodes, where each size includes 5 distinct graphs. The proposed framework is compared against Vanilla QAOA and ADAPT-QAOA in terms of both computational efficiency and solution quality.

6.5.1 Inference Time Analysis Across Graph Sizes

This section evaluates the computational efficiency of different methods as the graph size increases, with a primary focus on the proposed hybrid LLM-GNN framework. The mean inference time per run (in seconds) is reported for vanilla QAOA, ADAPT-QAOA, and all framework variants. The detailed numerical results are summarized in Table 6-6.

Nodes	Vanilla QAOA	ADAPT-QAOA	LLaMA-Feather	LLaMA-GNN	LLaMA-NetLSD	NanoGPT-Feather	NanoGPT-GNN	NanoGPT-NetLSD
4	3.362	1.565	0.237	0.221	0.210	0.132	**0.120**	0.124
6	9.600	0.807	0.177	0.205	0.185	0.101	**0.100**	0.103
8	32.043	1.973	0.199	0.200	0.732	0.114	0.121	**0.108**
9	65.250	5.663	0.197	0.172	0.167	**0.093**	0.104	**0.093**
10	84.892	9.135	0.195	0.179	0.184	**0.107**	0.115	0.110
11	167.564	23.600	0.183	0.169	0.166	0.105	0.105	**0.093**
12	258.761	69.190	0.218	0.166	0.149	0.098	**0.091**	0.092
13	381.595	210.725	0.426	0.132	0.130	0.094	**0.091**	**0.091**

Table 6-6 Mean Inference Time (s) per Method vs Graph Size for Vanilla QAOA, ADAPT-QAOA and This work's methods. Results Are Presented as Mean Values Across 5 Graphs For Each n nodes graph. The best and second-best values in each row are highlighted in **bold** and underlined, respectively. Methods proposed in this work are additionally indicated with a green background.

A clear distinction can be observed between quantum optimization methods and the proposed learning-based approach. Both vanilla QAOA and ADAPT-QAOA show a strong increase in inference time as the number of nodes grows. Specifically, vanilla QAOA increases from 3.362 seconds at 4 nodes to 381.6 seconds at 13 nodes, while ADAPT-QAOA rises from 1.565 seconds to 210.7 seconds over the same range. This trend is more clearly visible in Figure 6-30 and further emphasized in the logarithmic view in Figure 6-31.

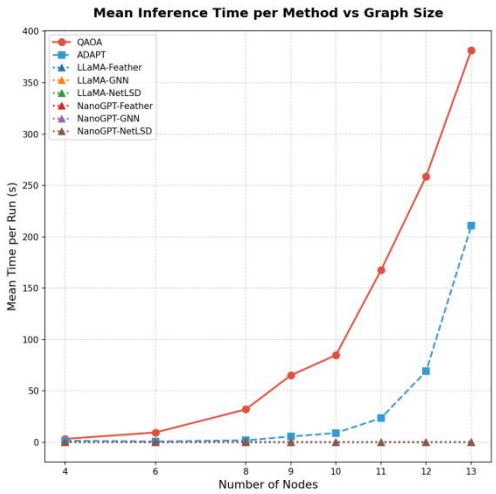

Figure 6-30 Mean Inference Time per Method vs Graph Size (Linear Scale)

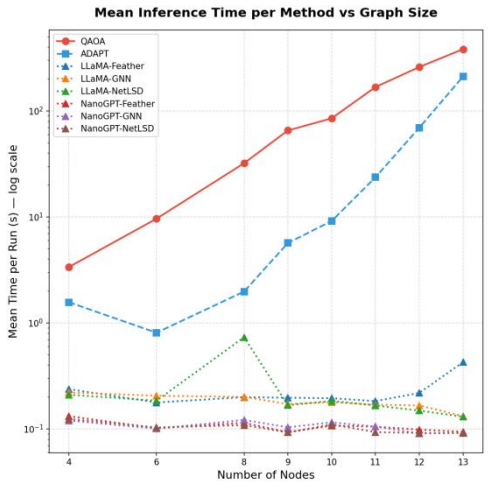

Figure 6-31 Mean Inference Time per Method vs Graph Size (Log Scale)

This behavior can be explained by the computational complexity of the two methods. Vanilla QAOA involves repeated evaluation of parameterized quantum circuits combined with classical optimization, leading to a complexity that scales at least polynomially with respect to the number of nodes and often approaches exponential behavior in practice due to the underlying 2^n state space. ADAPT-QAOA further increases overhead through iterative operator selection, requiring repeated gradient evaluations and parameter re-optimization. As graph size grows, the number of variables and candidate operators increases, resulting in a rapid rise in inference time.

Although QAOA and ADAPT-QAOA are executed on CPUs while the LLM-GNN framework runs on GPUs, this hardware difference does not fully explain the

trend. Even with this distinction, QAOA-based methods clearly show increasing runtime with graph size, whereas the LLM-based framework remains stable, indicating that the scaling behavior is mainly due to algorithmic complexity.

In contrast, the proposed hybrid LLM-GNN framework maintains nearly constant inference time (around 0.08-0.25 seconds) across all graph sizes, as it requires only a single forward pass through a trained model. Despite being trained on 11-node graphs, the models generalize efficiently without added cost, highlighting the benefit of shifting computation to offline training.

Among the variants, NanoGPT-based models are consistently faster than LLaMA-based ones due to their lighter architecture, though both remain stable across sizes. Additionally, the choice of graph embedding (FEATHER, GNN, or NetLSD) has minimal impact on runtime. Overall, while QAOA and ADAPT-QAOA scale poorly with graph size, the proposed framework achieves consistently low and stable inference time, offering a clear advantage for scalable applications.

6.5.2 Approximation Ratio Performance vs Graph Size

This section evaluates the solution quality of different methods as the graph size increases, measured by the approximation ratio (AR), with a primary focus on the proposed hybrid LLM-GNN framework. The mean approximation ratio per method is reported for vanilla QAOA, ADAPT-QAOA, and all framework variants. The detailed numerical results are summarized in Table 6-7 and visualized in Figure 6-32

From the results, ADAPT-QAOA consistently achieves the highest approximation ratios across all graph sizes, remaining close to optimal (approximately 0.96-0.99). Vanilla QAOA also shows stable performance, with AR gradually improving as graph size increases. These results confirm that QAOA-based methods, despite higher computational cost, maintain strong and reliable solution quality across different problem sizes.

Nodes	QAOA	ADAPT -QAOA	LLaMA -Feather	LLaMA-GNN	LLaMA-NetLSD	NanoGPT-Feather	NanoGPT-GNN	NanoGPT-NetLSD
4	0.766	**0.995**		0.668	0.835			0.741
6	0.837	**0.981**	0.751	0.758	0.748		0.700	0.760
8	0.891	**0.980**	0.650	0.649	0.659	0.658	0.646	0.647
9	0.869	**0.970**	0.657	0.658	0.665	0.673	0.655	0.657
10	0.885	**0.981**	0.704	0.725	0.694	0.727	0.680	0.727
11	0.876	0.963	0.955	0.947	0.957	0.972	0.965	**0.984**
12	0.878	**0.969**	0.751	0.751	0.750	0.716		0.721
13	0.906	**0.969**	0.763	0.767	0.766			0.750

Table 6-7 Mean Approximation Ratio (AR) per Method vs Graph Size for Vanilla QAOA, ADAPT-QAOA, and Proposed Framework Variants. Results are reported as mean values across 5 graphs for each n-node setting. Missing values indicate cases where valid circuits were not generated for all instances. The best and second-best values in each row are highlighted in **bold** and underlined, respectively. Methods proposed in this work are additionally indicated with a green background.

In contrast, the proposed LLM-GNN framework exhibits strong dependence on the training distribution. All variants achieve their best performance at 11 nodes, the size used during training, where AR values range from 0.947 to 0.984, with NanoGPT-based models slightly outperforming LLaMA-based ones. However, for other graph sizes, performance drops significantly, typically to the range of 0.65-0.76, indicating limited generalization.

An important observation from Table 6-7 is the presence of missing (null) values for several NanoGPT-based configurations at certain graph sizes. These indicate that the model failed to generate valid circuits for all 5 graph instances at those sizes. This suggests that while NanoGPT achieves the best peak performance at the trained size (11 nodes), it suffers from poorer generalization and robustness across unseen graph sizes. In contrast, LLaMA-based variants, although slightly weaker at 11 nodes, produce more consistent and valid outputs across a wider range of graph sizes.

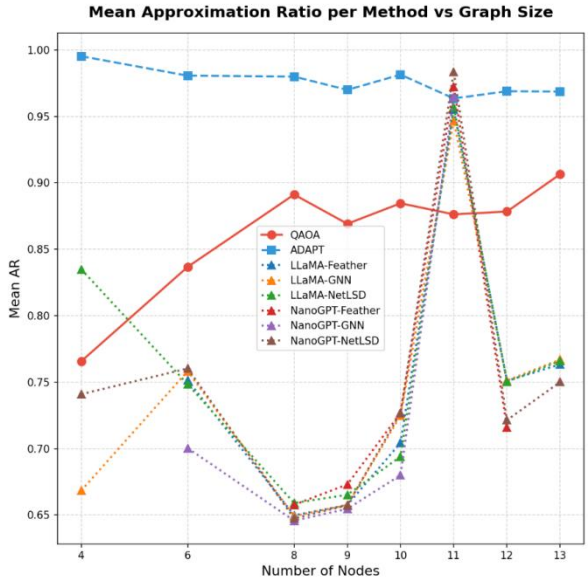

Figure 6-32 Mean Approximation Ratio (AR) per Method vs Graph Size

This highlights a key trade-off: NanoGPT appears more specialized and better fitted to the training distribution, whereas LLaMA demonstrates stronger generalization capability. Consequently, LLaMA-based models may have greater potential for improvement if trained on more diverse datasets.

Overall, the results indicate that the proposed framework is highly effective when applied to graph sizes similar to the training data but struggles to generalize beyond it. This limitation can be addressed in future work by constructing training datasets that include multiple graph sizes. Such an approach is expected to improve both robustness and generalization, enabling the framework to maintain high approximation ratios across a broader range of problem scales.

CHAPTER 7. CONCLUSIONS & FUTURE WORK

7.1 Conclusions

This thesis presents a hybrid LLM-GNN assisted ADAPT-QAOA framework for generating quantum circuits for combinatorial optimization problems. By combining graph representation learning with transformer-based sequence modeling, the framework learns to generate quantum circuits directly from graph structures, reducing the need for expensive iterative optimization. The study demonstrates that both NanoGPT and LLaMA architectures can successfully learn graph-to-circuit mappings, while different embedding methods provide varying trade-offs between stability and structural expressiveness.

Compared with traditional QAOA-based methods, the proposed framework offers significantly lower and more stable inference cost because circuit generation only requires a single forward pass through a trained model. Although ADAPT-QAOA still achieves the strongest optimization performance, the proposed method provides a more scalable and computationally efficient alternative while maintaining competitive solution quality on trained graph distributions.

However, the framework also has several limitations. Its performance strongly depends on the training distribution and decreases on unseen graph sizes, showing limited generalization capability. NanoGPT-based models achieve strong performance on trained graph sizes but are less robust across different sizes, while LLaMA-based models produce more stable outputs but with lower peak performance. In addition, the framework relies heavily on the diversity and quality of the training dataset and currently cannot fully replace adaptive optimization methods for problems requiring broad generalization.

Overall, this work shows that integrating graph neural representations with large language models is a promising direction for scalable quantum circuit design,

while also highlighting generalization and robustness as important challenges for future research.

7.2 Future Work

While this work demonstrates the effectiveness of the proposed LLM-GNN assisted QAOA framework, several important directions remain for further improvement and extension. These directions focus on improving scalability, learning objectives, and the integration between model components.

End-to-End Joint Learning of GNN and LLM Components. In the current framework, the GNN and LLM are trained separately, which may limit the alignment between graph representations and circuit generation. Future work should explore end-to-end training, allowing both components to be optimized together. This could produce more task-specific embeddings and improve overall circuit quality.

Task-Aware Training Objectives for Quantum Circuit Generation. The current model is trained mainly with autoregressive token prediction, which does not directly optimize quantum optimization performance. Future research can incorporate task-specific objectives such as approximation ratio and circuit validity into the training process through reinforcement learning or multi-objective optimization, enabling the model to generate more reliable and higher-quality circuits.

Generalization Across Problem Sizes. The framework currently relies on models trained for specific graph sizes, limiting scalability and flexibility. A future direction is to develop a unified model that can generalize across multiple graph sizes using size-invariant representations or curriculum learning strategies. This would reduce retraining requirements and improve practical applicability.

Scalability and Real-World Applicability. Future work should also focus on scaling the framework to larger graphs and adapting it to real quantum hardware constraints such as noise, limited connectivity, and gate restrictions. Extending the

framework to other combinatorial optimization problems and real-world datasets would further validate its effectiveness and broaden its practical use.

Together, these directions aim to transform the proposed framework into a more unified, scalable, and task-aware system, capable of addressing more complex quantum optimization challenges in practical settings

BIBLIOGRAPHY

[1] L. Huynh, J. Hong, A. Mian, H. Suzuki, Y. Wu, and S. Camtepe, "Quantum-Inspired Machine Learning: a Survey," Aug. 2023, [Online]. Available: http://arxiv.org/abs/2308.11269

[2] K. Zaman, A. Marchisio, M. A. Hanif, and M. Shafique, "A Survey on Quantum Machine Learning: Current Trends, Challenges, Opportunities, and the Road Ahead," Oct. 2023, [Online]. Available: http://arxiv.org/abs/2310.10315

[3] Y. Liao and C. Ferrie, "GPT on a Quantum Computer," Mar. 2024, [Online]. Available: http://arxiv.org/abs/2403.09418

[4] Tyagin, I., Farag, M. H., Sherbert, K., Shirali, K., Alexeev, Y., & Safro, I. (2025). "QAOA-GPT: Efficient Generation of Adaptive and Regular Quantum Approximate Optimization Algorithm Circuits." ArXiv. https://arxiv.org/abs/2504.16350

[5] Jiang, L., Zhang, C., & Chen, F. (2025). "QSeer: A Quantum-Inspired Graph Neural Network for Parameter Initialization in Quantum Approximate Optimization Algorithm Circuits." ArXiv. https://arxiv.org/abs/2505.06810

[6] Zhou, J., Cui, G., Hu, S., Zhang, Z., Yang, C., Liu, Z., Wang, L., Li, C., & Sun, M. (2018). "Graph Neural Networks: A Review of Methods and Applications." ArXiv. https://arxiv.org/abs/1812.08434

[7] Zhu, L., Tang, H. L., Barron, G. S., A., F., Mayhall, N. J., Barnes, E., & Economou, S. E. (2020). "An adaptive quantum approximate optimization algorithm for solving combinatorial problems on a quantum computer." ArXiv. https://arxiv.org/abs/2005.10258

[8] Wang, Z., Hadfield, S., Jiang, Z., & Rieffel, E. G. (2017). "Quantum Approximate Optimization Algorithm for MaxCut: A Fermionic View." Physical Review A, 97(2), 022304. https://doi.org/10.1103/PhysRevA.97.022304

[9] Liang, Z., Liu, G., Liu, Z., Cheng, J., Hao, T., Liu, K., Ren, H., Song, Z., Liu, J., Ye, F., & Shi, Y. (2024). Graph learning for parameter prediction of quantum approximate optimization algorithm. arXiv. https://doi.org/10.48550/arXiv.2403.03310

[10] Rozemberczki, B., & Sarkar, R. (2020). "Characteristic Functions on Graphs: Birds of a Feather, from Statistical Descriptors to Parametric Models." ArXiv. https://arxiv.org/abs/2005.07959

[11] Tsitsulin, A., Mottin, D., Karras, P., Bronstein, A., & Müller, E. (2018). "NetLSD: Hearing the Shape of a Graph." Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining. https://doi.org/10.1145/3219819.3219991

[12] Li, L., Li, J., Song, Y., Qin, S., Wen, Q., & Gao, F. (2023). Proactively incremental-learning QAOA. arXiv. https://doi.org/10.48550/arXiv.2311.02302

[13] Amosy, O., Danzig, T., Porat, E., Chechik, G., & Makmal, A. (2022). Iterative-Free Quantum Approximate Optimization Algorithm Using Neural Networks. ArXiv. https://arxiv.org/abs/2208.09888

[14] Jain, N., Coyle, B., Kashefi, E., & Kumar, N. (2021). Graph neural network initialisation of quantum approximate optimisation. ArXiv. https://doi.org/10.22331/q-2022-11-17-861

[15] Wang, H., Li, P., Chen, M., Cheng, J., Liu, J., & Chen, T. (2024). GroverGPT: A Large Language Model with 8 Billion Parameters for Quantum Searching. ArXiv. https://arxiv.org/abs/2501.00135

[16] Nakaji, K., Kristensen, L. B., Kemmoku, R., Campos-Gonzalez-Angulo, J. A., Vakili, M. G., Huang, H., Bagherimehrab, M., Gorgulla, C., Wong, F., McCaskey, A., Kim, J. S., Nguyen, T., Rao, P., Gao, Q., Sugawara, M., Yamamoto, N., & Aspuru-Guzik, A. (2024). The generative quantum eigensolver (GQE) and its application for ground state search. ArXiv. https://arxiv.org/abs/2401.09253

[17] Farhi, E., Goldstone, J., & Gutmann, S. (2014). A Quantum Approximate Optimization Algorithm. ArXiv. https://arxiv.org/abs/1411.4028

[18] Minaee, S., Mikolov, T., Nikzad, N., Chenaghlu, M., Socher, R., Amatriain, X., & Gao, J. (2024). Large Language Models: A Survey. ArXiv. https://arxiv.org/abs/2402.06196

[19] Touvron, H., Lavril, T., Izacard, G., Martinet, X., Lachaux, M. A., Lacroix, T., Rozière, B., Goyal, N., Hambro, E., Azhar, F., Rodriguez, A., Joulin, A., Grave, E., & Lample, G. (2023). LLaMA: Open and Efficient Foundation Language Models. ArXiv. https://arxiv.org/abs/2302.13971

[20] Open Quantum Computing. QAOA. https://github.com/OpenQuantumComputing/QAOA, 2026. Accessed: 2026-05-03.

[21] JuliaGPU. Adapt.jl. https://github.com/JuliaGPU/Adapt.jl, 2026. Accessed: 2026-05-03.